Scan 引擎

用户手册

仅供 2026 中国研究生创“芯”大赛·EDA 精英挑战赛参赛使用，未经许可不得转发、复制或用于其他用途。

文档说明

概述

本文档为 Scan 引擎的用户手册，重点介绍 DFT（Design for Testability，可测试性设计）扫描链插入的基本概念和 DFT 扫描链插入的使用方法，帮助用户正确地配置、使用 Scan 引擎。

读者对象

本文档主要适用于具备基本的 DFT 领域知识的专业技术人员。

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
须知	用于传递工具的安全警示信息。如不避免则可能会导致工具损坏、数据丢失、工具性能降低或其它不可预知的结果。
说明	对正文中重点信息的补充说明。

命令约定

在阅读本文档前，用户需要先了解如下文档中出现的 TCL 命令格式或者符号约定。

约定	示例	含义
斜体	read_file -verilog_files *test.json*	表示命令或参数的取值，用户需要根据实际情况适配的字段。
< >	*<timeplateName>*	表示用户指定的内容，<>内为参数取值或参数类型。

约定	示例	含义
\	create_pattern *test_pattern* \ -bypass_end *1* \ -timeplate *timeplate_test*	单条命令太长不方便阅读，使用“\”将命令换行。
|	wgl | verilog	参数取值分隔符。
{ }	-instances {<instance_name>}	在{}中以列举的方式把取值列出来，不同取值之间空格区分。

目录

文档说明 ii

1 Scan 引擎介绍 1

1.1 概述 2

1.2 在设计流程中的位置 2

1.3 支持的功能 2

1.4 输入与输出 3

1.4.1 输入文件 3

1.4.2 输出文件 4

1.5 调用 Scan 引擎 4

1.6 约束和限制 5

2 运行 Scan 引擎 6

2.1 运行 Scan 引擎的基本流程 7

2.1.1 读入文件 8

2.1.2 设置 Scan 参数 10

2.1.2.1 指定当前设计 10

2.1.2.2 定义 DFT Signal 11

2.1.2.3 识别已有插链信息 20

2.1.2.4 配置 Scan Chain 23

2.1.2.5 配置 Clock-Gating 27

2.1.2.6 配置 Wrapper Chain 28

2.1.2.7 配置 DFT Partition 38

2.1.2.8 配置 IP Group 的 Wrapper Chain 43

2.1.2.9 配置 Scan Replacement 46

2.1.3 执行 DRC 48
2.1.3.1 DRC 规则 48
2.1.3.2 查询 DRC 信息 51
2.1.3.3 调整 DRC 违例等级 53
2.1.3.4 Design Learning 55
2.1.4 预生成及插入 Scan Chain 57
2.1.4.1 预生成 Scan Chain 57
2.1.4.2 插入 Scan Chain 57
2.1.5 输出文件 58
2.1.5.1 输出插链信息 58
2.1.5.2 输出网表 63
2.1.5.3 输出 CTL 文件 64
2.1.5.4 输出 SCANDEF 文件 64
2.2 Scan 引擎支持场景及 Dofile 示例 65
2.2.1 替换寄存器 65
2.2.2 仅连接 ICG 66
2.2.3 插入 Scan Chain 67
2.2.3.1 基本场景 68
2.2.3.2 读取 CTL 68
2.2.3.3 DFT Partition 69
2.2.4 对多个设计进行 Scan 插入并集成 71
3 附录 73
3.1 特殊场景说明 74
3.1.1 Wrapper 分析与用户自定义 Chain 存在冲突时的处理策略说明 74
3.2 术语表 77
3.2.1 A-E 77
3.2.2 F-J 79
3.2.3 K-O 80
3.2.4 P-T 81
3.2.5 U-Z 83

1 Scan 引擎介绍

1.1 概述

1.2 工作流程

1.3 支持的功能

1.4 输入与输出

1.5 调用 Scan 引擎

1.6 约束和限制

1.1 概述

随着集成电路特征尺寸的不断缩小以及芯片面积和晶体管数量的不断增加，在芯片制造过程中引入缺陷的概率也相应提高。因此，芯片在出厂前必须经过严格的测试，避免将有缺陷的芯片交付给客户。为了满足这一需求，可测试性设计技术（DFT，Design for Testability）应运而生并广泛应用，使芯片在设计阶段就能够兼顾其可测试性，也有效控制了快速攀升的测试成本。

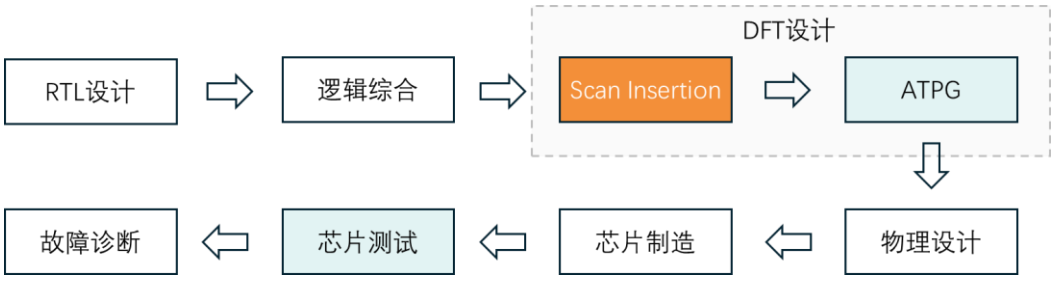
Scan Insertion 是 DFT 技术中最基础且关键的一环。其目标是在不改变电路原有功能逻辑的前提下，尽可能多地将电路中的时序器件替换为对应的扫描器件，并连接成扫描链，同时暴露必要的控制和数据端口。芯片制成后，测试设备便可以经这些测试端口写入测试向量、控制电路的内部状态，从而显著提升电路的可测试性。

Scan 引擎可实现 Scan Insertion 的功能，本章将介绍 Scan Insertion 在设计流程中的位置、Scan 引擎支持的功能、输入输出、调用方式及限制约束。

1.2 在设计流程中的位置

Scan Insertion 应用于芯片设计流程的 DFT 设计阶段，在逻辑综合之后、物理设计之前。Scan Insertion 会在芯片原有的功能逻辑之上，完成测试基础设施的铺建，进而将扫描链及其端口信息传递给 ATPG 工具生成测试计划和测试向量。Scan Insertion 在设计流程中的位置如下图所示。

图 1-1 Scan Insertion 在设计流程中的位置



1.3 支持的功能

Scan 引擎支持的主要功能如表 1-1 所示：

表 1-1 Scan 引擎支持的功能

功能	描述
Scan Chain	支持根据配置完成电路中时序器件的替换和 Scan Chain 插入。
	支持多场景的 Scan Insertion 流程，包括 Scan 寄存器替换、ICG TE 端的连接、Scan Chain 插入。
	支持分 Partition 插入 Scan Chain。
	支持读入 CTL 文件，解析设计中已有的插链结构。
	支持对电路执行 Wrapper 分析和 Wrapper 插链。
	支持生成 Scan 网表，SCANDEF 及 CTL 文件。
DRC	支持对插链前网表的设计规则检查（DRC），将输入电路中可能会阻碍、影响后续可测试性设计的位置提前报出，供用户确认、诊断、修复。
	支持 DRC Gui 界面显示，并进行 Debug。

1.4 输入与输出

Scan 引擎输入输出文件如下表所示。

表 1-2 Scan 引擎输入与输出

输入输出类型	文件类型	文件格式	支持版本类型
Input	Netlist	Verilog	95、91、05
	Cell library	Liberty	-
	Test model	CTL	IEEE
Output	Netlist	Verilog	95、91、05
	Test model	CTL	IEEE
	SCANDEF	SCANDEF	v5.8

1.4.1 输入文件

1. 网表文件（Netlist）

Scan 引擎需要用户提供 Verilog 格式的门级网表文件作为输入。用户若导入其它格式的网表文件，引擎将忽略这些网表及网表中的内容，仅加载解析 Verilog 格式的网表文件。该输入文件为必选文件。

2. 库文件（Cell Library）

   Scan 引擎需要用户提供 Liberty 格式（.lib）的库文件作为输入。若已在综合阶段读入文件，则可跳过该步骤。该输入文件为必选文件。

3. 测试模型文件（Test Model）

   Scan 引擎允许用户提供 CTL（ASCII 格式）的测试模型文件作为输入，用来描述设计中已有的插链结构。对于已提供 CTL 文件的 Module，仍需读入该设计的网表文件。该输入文件为可选文件。

1.4.2 输出文件

1. 网表文件（Netlist）

   Scan 引擎为用户提供插链后的 Verilog 门级网表文件作为输出。

2. 测试模型文件（Test Model）

   Scan 引擎为用户提供 CTL 格式的测试模型文件作为输出，用来描述网表文件中已有的插链结构。

3. SCANDEF 文件

   Scan 引擎为用户提供 SCANDEF 文件作为输出。

1.5 调用 Scan 引擎

用户启动工具后首次调用任何一条 Scan 引擎的命令，都会触发 Scan 引擎的 License 校验。此时系统会要求用户输入 License 授权信息。如果 License 校验不通过，则退出校验机制，Scan 引擎调用失败；如果 License 校验通过，即视为成功调用 Scan 引擎，用户可在工具内继续调用 Scan 引擎的其它命令。Scan 引擎在检测到 License 有效时间不足 30 天时会给出告警。

说明

- Scan 引擎支持通过 Source 命令加载并执行 Dofile 文件，该文件中可包含多条 Scan 引擎指令。用户可在 Dofile 文件的首行添加如下命令：

  ```
  set_parameter global.sh_continue_on_error true
  ```

  添加该命令后，Scan 引擎在执行 Dofile 过程中，即使某条命令因 ERROR 级别错误执行失败，也不会影响后续命令的执行。

若用户未在 Dofile 中添加该命令，Scan 引擎在遇到 ERROR 错误时将立即停止执行后续命令。

- 作为工具的组成部分，Scan 引擎兼容工具的命令体系。例如，由于工具的 load_db 命令无法兼容旧版本的数据库文件，因此 Scan 引擎所调用的该命令同样不支持旧版文件格式。

1.6 约束和限制

Scan 引擎的限制项说明如下：

1. 不支持将逻辑单元集合识别为集成门控时钟单元（Integrated Clock Gating，ICG）。例如，对于网表中由 Latch、AND、OR 等逻辑单元组成的具有 ICG 相似功能的逻辑单元集合，Scan 引擎不会将其识别为 ICG。
2. 仅支持最基本的三态设计（一个数据端 I、一个控制端 OE、一个输出 Z），不支持其他复杂类型的三态设计。
3. 网表和标准单元库的加载、解析、处理包括以下限制：
   - 不支持读取 CTL 文件中以 Pseudo Port 及 Hookup Pin 定义的 DFT Signal；
   - 不支持识别网表中缺少例化名的 Cell。例如，由于缺少例化名，下例网表中的 *cell2* 将不会被加载解析，读取网表时将忽略此 Cell 例化的 Instance。

     ```
     cell1 inst1
     cell2
     ```

     其中 *cell1*、*cell2* 为标准单元名，*inst1* 为标准单元的例化名。

2 运行 Scan 引擎

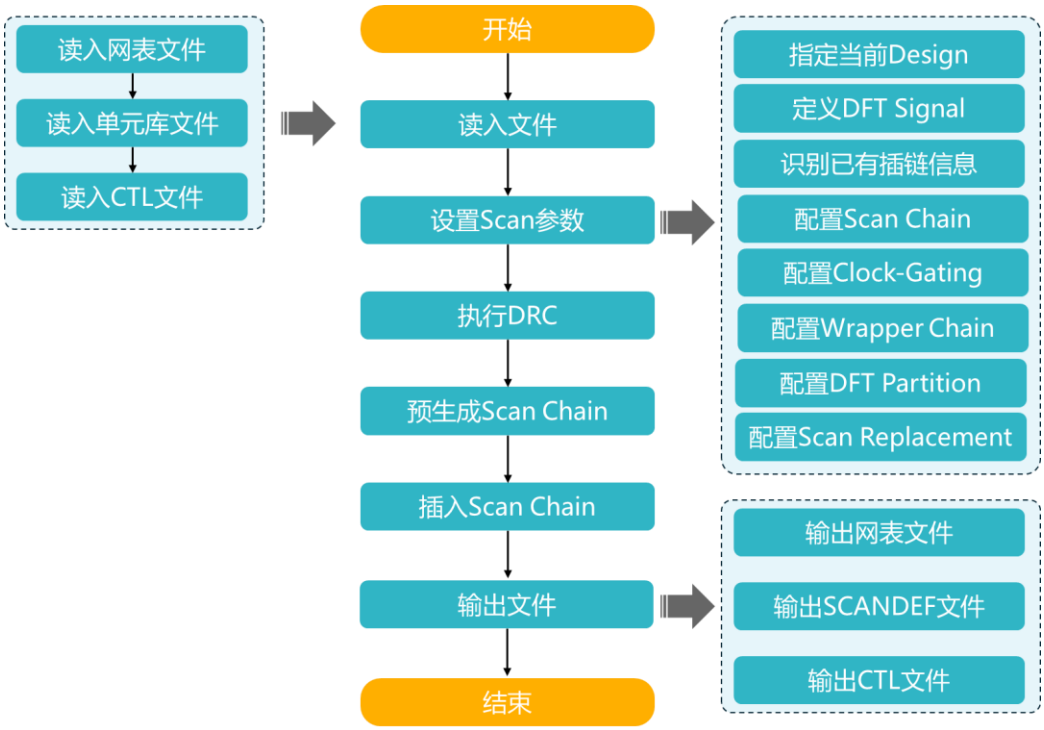
2.1 运行 Scan 引擎的基本流程

2.2 Scan 引擎支持场景及 Dofile 示例

2.1 运行 Scan 引擎的基本流程

运行 Scan 引擎的基本流程适用于大多数设计。要运行 Scan 引擎，用户需要提供输入文件以及定义运行 Scan 引擎过程中的各项设置。

图 2-1 运行 Scan 引擎的基本流程



运行 Scan 引擎的流程包括以下步骤：

1. 读入文件

   读入综合后的网表文件、单元库文件和 CTL 文件。若已在综合阶段读入文件，则可跳过该步骤。更多信息，请参考 2.1.1 读入文件章节。

2. 设置 Scan 参数

   在读入文件后，需要指定设计的顶层。在指定顶层设计后，为了生成 Scan Chain，用户需要对插链参数进行一些设置，其中包括定义 DFT Signal、识别已有插链信息、配置 Scan Chain、配置 Clock-Gating、配置 Wrapper Chain、配置 DFT Partition、配置 Scan Replacement。更多信息，请参考 2.1.2 设置 Scan 参数章节。

3. 执行 DRC

   在设置 Scan 参数以后，需要执行 DRC，执行 DRC 的过程中会触发 Design Learning。更多信息，请参考 2.1.3 执行 DRC 章节。

4. 预生成 Scan Chain

   在执行 DRC 以后，需要分析当前设计中的 Scan Cell 及 Wrapper Cell。接着按照配置规则和 Order 算法，将设计中的 Scan Cell 及 Wrapper Cell 分配到 Scan Chain 及 Wrapper Chain 上，最终预生成 Scan Chain 及 Wrapper Chain。更多信息，请参考 2.1.4 预生成及插入 Scan Chain 章节。

5. 插入 Scan Chain

   在预生成 Scan Chain 以后，可以将预生成的 Scan Chain 及 Wrapper Chain 插入网表中。更多信息，请参考 2.1.4 预生成及插入 Scan Chain 章节。

6. 输出文件

   输出 Scan-Inserted 网表，SCANDEF 和 CTL 文件。更多信息，请参考 2.1.5 输出文件章节。

2.1.1 读入文件

运行 Scan 引擎需要读入综合后的网表和单元库文件，若运行 Scan 引擎前的设计中已存在插链信息，则需读入 CTL 文件。

1. 读入网表文件

   网表文件描述了芯片设计电路元件相互之间的连接关系，Scan 引擎支持读入.v 格式的网表文件和.gz 格式的压缩网表文件。用户可使用以下命令读入网表文件：

   ```
   load_netlist ./test.v
   ```

   其中，*test.v* 为读入的网表文件的名称。

   说明

   在读入网表信息的过程中，

   - 支持在一次命令中读入多份网表文件；
   - 若已在综合阶段读入网表文件，则可在 Scan 引擎中跳过该步骤。

2. 读入单元库文件

   单元库文件包含了组合逻辑、时序逻辑、功能单元和特殊类型单元。Scan 引擎支持读入.lib 格式的 Liberty 单元库文件和.gz 格式的压缩单元库文件。用户可用以下命令读入库文件：

   ```
   load_lib {./test_1.lib ./test_2.lib }
   ```

   其中，test_1.lib 和 test_2.lib 为读入的单元库文件的名称。

   说明

   在读入单元库文件的过程中，

- 支持在一次命令中读入多份单元库文件；
- 若已在综合阶段读入单元库文件，则可在 Scan 引擎中跳过该步骤。

3. 读入 CTL 文件

Scan 引擎支持输入 ASCII 格式的 CTL 文件。用户可用以下命令读入 CTL 文件：

```
load_ctl -module core1 ./test.ctl
```

其中，*test.ctl* 为读入的 CTL 文件的名称。

CTL 文件记录了设计的扫描链信息。通过导入子设计对应的 CTL 文件，可实现将子设计的扫描链结构集成到顶层设计中。CTL 文件分为四个部分：表头、设计配置信息、Scan 结构信息和测试信息。其中表头、设计配置信息和 Scan 结构信息为必要内容。以下分别介绍各部分的内容以及示例。

- 表头部分包括版本号、设计名及日期等信息。该部分示例如下：

```
STIL 1.0 {
CTL P2001.10;
Design 2005;
}
Header {
Title "xxx";
Date "xxx";
Source "xxx";
}
```

- 设计配置信息包含以下内容：端口名称、端口方向、Scan 时钟信号名称及极性、异步置位/复位信号、及其他测试相关信号。该部分示例如下：

```
Signals {
"clk" In;
"rst" In;
"data" In;
"se" In;
"datal " Out;
"test_si" In;
"test_so" Out;
}
…
Environment top_design {
CTL Internal_scan {
Internal {
     "test_si" {
       CaptureClock  "clk" {
         LeadingEdge;
       }
      }
     "test_so" {
       LaunchClock  "clk" {
         LeadingEdge;
     …
       }
      }
```

```
    }
  }
}
```

- Scan 结构信息包含以下内容：Scan Chain 类型、Scan Chain 名称、Scan Chain 长度、Scan Chain 上的 Scan Cell 名称、Scan Chain 的信号端口类型与名称以及链末尾 Lockup Latch 等信息。

```
ScanStructures Internal_scan {
  ScanChain “chain1” {
ScanLength 2;
ScanCells
 “scancell1”
 “scancell2” ;
ScanIn “test_si” ;
ScanOut “test_so” ;
ScanEnable “se” ;
ScanMasterClock “clk” ;
}
  ScanChain “chain2” {
…
  }
}
```

说明

仅支持用户在指定当前设计前执行以上读入文件命令。如果用户想在指定当前设计后读入文件，需重新指定当前设计。

2.1.2 设置 Scan 参数

2.1.2.1 指定当前设计

读入文件后需要指定当前的设计。用户可使用以下其中一条命令指定当前设计：

```
load_netlist xxx.v -top design_name1
present_design design_name1
```

其中 *design_name1* 为用户指定的当前设计名称，该名称必须和网表文件中的一个 Module 名称一致。

若用户没有指定当前设计，Scan 引擎会默认将网表文件中的 Top Module 指定为当前设计。若网表中存在多个 Top Module，Scan 引擎会按照 Module 的读取顺序，将首个 Top Module 指定为当前设计。用户设置的 Scan 参数只对当前设计生效。

用户可以多次指定不同的设计，但只有最后一次指定的设计会生效。同时，Scan 引擎将删除之前指定设计的所有相关配置。如果用户指定的设计名称和网表文件中的 Module 名称不一致，Scan 引擎将报错并保持当前设计。Scan 引擎不支持用户将网表文件中的 Sub Module 指定为当前设计。

须知

Scan 引擎从当前设计切换到其他设计时，不保留当前设计设置的 Scan 参数。

2.1.2.2 定义 DFT Signal

Scan 引擎支持的 DFT Signal 包括以下类型：测试时钟信号、置位/复位信号、测试使能信号、以及测试数据信号。

定义测试时钟信号

Scan 引擎在插链前必须设置 Scan 时钟信号，以确定哪些时序单元（Sequential Cell）符合插链条件。因此，在 Scan Insertion 流程中必须对这些 Scan 时钟信号进行定义。Scan 时钟分为以下两类：

- 源时钟（Source Clock）

  源时钟为定义在原始端口（Primary Port）或伪端口（Pseudo Port）上的时钟信号。用户可以使用 **set_scan_signal** 命令设置时钟信号。例如将时钟定义在端口 *port_name1* 上，并指定时钟的关闭状态（off_state）值为 0，示例如下：

  ```
  set_scan_signal -type clock -port port_name1 -off_state 0
  ```

  其中，**type** 表示定义的 Scan 时钟的类型。用户必须将 Scan 时钟的类型指定为 *clock*。**port** 表示指定的 Scan 时钟的端口。用户指定的端口必须为网表中实际存在的端口。端口名即为定义的 Scan 时钟信号的名称，例如上述命令中定义的 Scan 时钟名称为 *port_name1*。

  说明

  Scan 时钟的端口必须存在于网表中。用户如需定义多个 Scan 时钟信号，必须在不同的端口上定义。

  在 Wrapper Flow 中，用户必须为 Dedicated Wrapper Cell 设置 Wrapper 时钟。设置 Wrapper 时钟的时候，用户必须指定 Wrapper 时钟的类型和端口。如果用户指定的端口不存在于网表中，Scan 引擎将默认在当前设计中创建该端口。Wrapper 时钟在当前设计中只可存在一个。如果用户在多个端口上设置了 Wrapper 时钟，以最后一次指定的 Wrapper 时钟的端口为生效端口。

  用户可以使用 **rpt_scan_signal** 命令报出当前设计中的时钟信号。

- 内部时钟（Internal Clock）

  内部时钟为定义在电路内部引脚（Pin）上的时钟信号。用户可使用以下命令设置当前设计中的内部时钟：

  ```
  set_scan_signal -type clock -port port_name1 -internal_clocks single
  ```

其中，*port_name1* 为指定时钟的端口名称，*single* 为划分内部时钟的其中一种类型。该命令表示在端口 *port_name1* 之后经过单输入逻辑门后的时钟域被视为单独的内部时钟域。

说明

通过该命令生成的内部时钟的波形（Waveform），需要根据用户定义在端口上的时钟往后追踪相应的逻辑门电路计算得到。示例中划分内部时钟类型分以下情况：

- none：不识别内部时钟域；
- single：将任意一个经过任意组合逻辑门（包括 buffer/inverter）后的时钟域视为单独的内部时钟域；
- multi：将任意一个将经过多输入组合逻辑门后的时钟域视为单独的内部时钟域。

用户可以使用以下命令将当前设计中的内部引脚定义为内部时钟域的起始点：

```
set_scan_signal -type clock -port port_name1 -associated_internal_clocks pin_name1
```

其中，*port_name1* 为指定时钟的端口名称，*pin_name1* 为指定内部时钟的引脚名称。

说明

- 用户定义在内部引脚上的内部时钟的波形和定义在端口上的源时钟的波形完全一致，即内部时钟的关闭状态与在源时钟的关闭状态相同；
- 命令 set_scan_signal -internal_clocks 和 set_scan_signal -associated_internal_clocks 的执行顺序不会影响内部时钟的创建结果。内部时钟的创建结果取两条命令的去重合集，当冲突时取 set_scan_signal -associated_internal_clocks 命令定义的波形。

创建内部时钟的示例如下：

```
set_scan_signal -type clock -port CLK1 -off_state 0 -associated_internal_clocks /core1/U1/pin1
set_scan_signal -type clock -port CLK1 -off_state 0 -internal_clocks single
```

设置置位/复位信号

用户可以使用以下命令设置置位/复位信号：

```
set_scan_signal -type reset
```

添加置位/复位类型信号时，必须由用户显式指定。用户可在命令中使用**-port** 参数，指定当前设计中的置位/复位信号，指定的端口必须为当前设计中已存在的输入类型的端口。用户可以在命令中使用**-off_state** 命令，定义置位/复位信号的关闭状态（off_state）的值。默认关闭状态的值为 0，即生效值为 1。

设置测试使能信号

测试使能信号根据 Chain 的类型分为以下两类：Scan Chain 相关的测试使能信号和 Wrapper Chain 相关的测试使能信号。以下分别对其介绍：

- Scan Chain 相关的测试使能信号

  用户可以使用以下命令设置 Scan Chain 相关的测试使能信号：

  ```
  set_scan_signal -type scan_enable
  ```

  如果用户不使用该命令指定使能信号，Scan 引擎在执行 insert_dft_logic 命令后将自动创建必要的使能信号。

  用户可在该条命令中使用**-usage** 参数项，对当前设计中的 Clock-Gating 和 Functional FF 分别指定若干多个 Scan 使能信号。设置的 Clock-Gating 和 Functional FF 将在插链后（**insert_dft_logic**）的网表中连接上指定的 Scan 使能信号。

  用户可通过以下命令，设置使能信号用于插链后新生成的 Scan Chain 上的 Scan Cell：

  ```
  set_scan_signal -type scan_enable -view spec -port port_name1 -off_state 0 -usage all
  ```

  其中，参数项-view 仅支持和-type scan_enable 同时使用。该参数项的作用是区分该 scan_enable 信号是网表中已经存在连接的 scan_enable 信号还是用户指定用于插链的信号。不指定该参数项时取默认值 *spec*。*spec* 指测试使能信号用于插链后新生成的 Scan Chain 上的 Scan Cell；*port_name1* 为测试使能信号的端口名称，关闭状态值为 0；*all* 表示将该测试使能信号指定给当前设计中所有即将插链的 Clock-Gating 和 Functional FF。

  用于新生成链的测试使能信号可存在多个端口，**-usage** 值相同的以最后一次指定的为准。

  用户可以指定网表中已存在且先前被定义过的测试使能信号。例如，定义网表中已存在且先前被定义过的 Scan Chain 和 Scan Segment 的测试使能信号。示例如下：

  ```
  set_scan_signal -type scan_enable -view existing -port port_name1 -off_state 0
  ```

  一个端口可以同时被指定为用于新生成链的测试使能信号和网表中已存在且先前被定义过的测试使能信号，且两者关闭状态（off_state）的值可以不同。网表中已存在且先前被定义过的 Scan 使能信号可定义在多个端口上。

  用户可以使用 **rpt_scan_signal** 命令报出当前设计中的测试使能信号。

- Wrapper Chain 相关的测试使能信号

  Wrapper Chain 相关的测试使能信号包括以下四类：wrp_in_shift_en、wrp_out_shift_en、wrp_in_capture_en 以及 wrp_out_capture_en。

- 设置 Wrapper Shift Enable 信号

  用户可以使用 **set_scan_signal** 命令指定当前设计中 Wrapper Chain 的 Wrapper Shift Enable 信号，该信号根据 Wrapper Chain 分为输入（Input）和输出（Output）类型。示例如下：

  ```
  set_scan_signal -type wrp_in_shift_en -port port_name1
  set_scan_signal -type wrp_out_shift_en -port port_name2
  ```

  其中，*port_name1* 和 *port_name2* 分别为 wrp_in_shift_en 和 wrp_out_shift_en 信号对应的端口名称。

- 设置 Wrapper Capture Enable 信号

  用户可以使用 **set_scan_signal** 命令为当前设计的 Dedicated Wrapper Cell 设置 Wrapper Capture Enable 信号，该信号根据 Wrapper Chain 分为输入（Input）和输出（Output）类型。示例如下：

  ```
  set_scan_signal -type wrp_in_capture_en -port port_name1
  set_scan_signal -type wrp_out_capture_en -port port_name2
  ```

  其中，*port_name1* 和 *port_name2* 分别为 wrp_in_capture_en 和 wrp_out_capture_en 信号对应的端口名称。

  说明

  用户在指定 Wrapper 相关的测试使能信号端口时，需注意以下事项：

  - 仅支持指定当前设计中输入类型的端口。如果用户指定了输出类型的端口，Scan 引擎将报错；
  - 如果用户指定了当前设计中不存在的端口，Scan 引擎将创建该端口并使用该端口作为 Wrapper 相关的测试使能信号端口；
  - 如果用户没有指定端口，Scan 引擎会在需要时自动新建端口作为 Wrapper 相关的测试使能信号端口；
  - 通过 **set_scan_signal** 命令指定的 Wrapper 测试使能信号端口，可作为 **set_wrapper_cfg** 命令中显示指定的 Wrapper 测试使能信号端口。

- 设置测试使能信号的实际连接点

  用户可指定测试使能信号在插链时的实际连接点，即指定测试使能信号的 Hookup Pin。在指定 Hookup Pin 时，命令参数**-port** 所指定的端口（Primary Port 或 Pseudo Port）必须存在于当前设计中。如果端口不存在，Scan 引擎不会新建该端口并会报错。用户可以通过参数**-hookup_sense** 定义 Hookup Pin 的值与端口定义的值之间的关系。例如，将测试使能信号定义在 Hookup Pin *pin_name1* 上，并指定 Hookup Pin 上信号的极性与定义在端口上的测试使能信号极性相同。示例如下：

  ```
  set_scan_signal -type scan_enable -port port_name1 -off_state 0 -hookup_pin pin_name1 -
  hookup_sense positive
  ```

其中，*port_name1* 为测试使能信号对应的端口名称，信号的关闭状态值为 0；*pin_name1* 为该测试使能信号在插链时的实际连接点的引脚名称；*positive* 指 Hookup Pin 上信号的极性与由端口定义的测试使能信号的极性相同，即 Hookup Pin 点的信号的关闭状态值为 0。

设置测试数据信号

测试数据信号根据 Chain 的类型分为以下两类：Scan Chain 相关的测试数据信号和 Wrapper Chain 相关的测试数据信号。以下分别对其介绍：

- Scan Chain 相关的测试数据信号

  Scan Chain 相关的测试数据信号包括：scan_data_in 和 scan_data_out。用户可以使用 **set_scan_signal** 命令指定用于当前设计中新生成的 Scan Chain 的测试数据信号。信号对应的端口名称默认为以下格式：test_si%d 和 test_so%d。示例如下：

  ```
  set_scan_signal -type scan_data_in -port test_si%d
  set_scan_signal -type scan_data_out -port test_so%d
  ```

  其中，*test_si%d* 为 scan_data_in 信号对应的端口名称；*test_so%d* 为 scan_data_out 信号对应的端口名称。

- Wrapper Chain 相关的测试数据信号

  Wrapper Chain 相关的测试数据信号包括：wrp_data_in 和 wrp_data_out。用户可以使用 **set_scan_signal** 命令指定用于当前设计中新生成的 Wrapper Chain 的测试数据信号。信号对应的端口名称默认为以下格式：wrp_si%d 和 wrp_so%d。示例如下：

  ```
  set_scan_signal -type wrp_data_in -port wrp_si%d
  set_scan_signal -type wrp_data_out -port wrp_so%d
  ```

  其中，*wrp_si%d* 为 wrp_data_in 信号对应的端口名称；*wrp_so%d* 为 wrp_data_out 信号对应的端口名称。

用户可以使用 **rpt_scan_signal** 命令报出当前设计中的测试数据信号。

报出 DFT Signal

用户可以使用 **rpt_scan_signal** 命令报出定义的 DFT Signal。示例如下：

```
rpt_scan_signal

Design: top
--------------------------------------------------------------------------------------------------------------------
Port PortProperty SignalType OffState HookupPin HookupSense AssociatedInternal Usage View ConstantValue OwnerPartition
--------------------------------------------------------------------------------------------------------------------
clk1  pre_existing  clock        0                          /core1/U2/pin3
```

```
clk2  pre_existing  clock         1
SE1   tool_created  scan_enable   0       U1/pin1  positive                    scan  spec            Default_Partition
SE2   pre_existing  scan_enable   0                                       clock_gating existing
int_pin1  pseudo    wrapper_clock 0
c1    pre_existing   constant                                                          1
SE_p1 tool_created  scan_enable   0                                          scan  spec               P1
--------------------------------------------------------------------------------------------------------------------------
```

报告的内容说明如下表所示：

表 2-1 DFT Signal 报告内容说明

表头字段	内容说明
Port	DFT Signal 的端口名。
Port_property	端口的属性，分为以下 3 类： • pre_existing：表示该端口为设计中已存在的原始端口； • tool_created：表示该端口为 Scan 引擎新创建的新端口； • pseudo：表示该端口为用户将引脚指定成的伪端口。
Signal_type	DFT Signal 的信号类型。
Off_state	DFT Signal 的关闭状态值。
Hookup_pin	DFT Signal 指定的 Hookup Pin 的引脚名。
Hookup_sense	DFT Signal 指定的 Hookup Pin 和端口上信号关闭状态值的对应值。参数取值为 positive 或 negative。 • positive：表示两者关闭状态值相同； • negative：表示两者关闭状态值相反。
Associated_internal	DFT Signal 上指定的内部时钟的起始点。
Usage	DFT Signal 中测试使能信号的使用对象。支持以下取值： • clock_gating：表示该测试使能信号仅用于 clock-gating 的连接； • scan：表示该测试使能信号仅用于 Functional FF 的连接； • all：表示该测试使能信号用于 clock-gating 和 Functional FF 的连接。

表头字段	内容说明
View	DFT Signal 中测试使能信号的创建属性。支持以下取值： • spec：表示用户指定用于将要创建的 Scan Chain 的信号； • existing：表示网表中已经存在连接的测试使能信号。
Constant_value	DFT Signal 中 constant 类型信号的约束常值。
OwnerPartition	DFT Signal 归属的 Partition。

删除 DFT Signal

用户可以使用 remove_scan_signal 命令删除定义的 DFT Signal。示例如下：

```
set_scan_signal -view spec -type scan_enable -port SE -off_state 0
set_scan_signal -view spec -type scan_enable -port SE_1 -off_state 0
set_scan_signal -view spec -type scan_enable -port SE_2 -off_state 0
...
remove_scan_signal -port {SE_1 SE_2}
```

其中 *port SE_1* 和 *port SE_2* 是被 set_scan_signal 指定成功的 Port Name（必须条件）。

说明

被 remove_scan_signal-port 指定的 Port，其上的所有 Signal 信息会被完全消除。

设置电路行为

- 定义电路输入端口及约束值

  在某些场景下，用户可能需要将内部信号标记为逻辑上可直接控制的输入端口（Pseudo Input），此时需要使用 **add_pseudo_pi** 命令。例如将内部信号 *u1/dff0/D u2/dff2/Q INV_0/Y* 标记为 Pseudo Input。示例如下：

  ```
  add_pseudo_pi {u1/dff0/D u2/dff2/Q INV_0/Y}
  ```

  该命令定义的对象支持输入类型和输出类型的 Pin List、Hierarchy Pin 以及 Leaf Pin。

  对于用户定义的 Pseudo Input，可通过 **rpt_pseudo_pi** 命令报出。示例如下：

  ```
  rpt_pseudo_pi
  Design: demo_case
  ---------------------
  PseudoPIName
  ---------------------
  u1/dff0/D
  u2/dff2/Q
  ```

```
INV_0/Y
--------------------
```

用户如果要对指定的输入增加约束，需要使用 **set_scan_signal -type constant** 命令指定端口名称及其约束值。端口的约束值可设为如下值之一：

- 常数 0（0）
- 常数 1（1）
- 高阻抗（Z）
- 未知（X）

例如，将端口 *port_name1* 和 *port_name2* 分别约束为 1 和 0。示例如下：

```
set_scan_signal -type constant -port port_name1 -constant_value 1
set_scan_signal -type constant -port port_name2 -constant_value 0
```

set_scan_signal -type constant 定义的对象仅支持输入类型的端口，包括原始输入端口和伪输入端口。

用户可以使用 **rpt_scan_signal** 命令报出当前设计下用户对电路施加的约束值。

- 定义 Scannable/Non-Scannable Cell

  用户可通过 **set_scan_element** 命令将时序单元标记为 Scannable Cell 或 Non-Scannable Cell。

  指定为 Scannable 的时序单元将参与 DRC 检查，检查通过的时序单元参与 Scan 替换及 Scan Chain 配置；检查不通过的时序单元不参与 Scan 替换与 Scan Chain 配置。该命令的作用范围包含 Hierarchy Cell 及 Leaf Cell。指定 Hierarchy Cell 时包括 Hierarchy 下所有的时序单元。命令示例如下：

  ```
  set_scan_element true {dff1 dff2}
  ```

  其中，*dff1* 和 *dff2* 为定义为 Scannable Cell 的寄存器名称。

  指定为 Non-Scannable 的时序单元不参与 DRC 检查，不参与 Scan 替换及 Scan Chain 配置。该命令作用范围包含 Hierarchy Cell 及 Leaf Cell。指定 Hierarchy Cell 时包括 Hierarchy 下所有的时序单元。命令示例如下：

  ```
  set_scan_element false {dff1 dff2}
  ```

  其中，*dff1* 和 *dff2* 为定义为 Non-Scannable Cell 的寄存器名称。

  说明

  使用 set_scan_element 命令时，需注意以下事项：

  - 该命令可以设置多次，最终的 instance_list 是所有设置叠加的结果；
  - 不支持将具有 dont_touch 属性 DFF 设置为 Scannable Cell；
  - 对已被 Design Learning 识别为 Non-Scannable 的 Cell，不支持设置为 Scannable Cell；
  - 对已被 DRC 检查中判定为 ERROR 违例的 Cell，不支持设置为 Scannable Cell；

- 对具有 dont_touch 属性的 DFF 执行 set_scan_element true 命令无效。此类 DFF 不支持配置为 Scannable Cell。

用户可以使用 **rpt_scan_element** 命令报出当前设计下 Scannable 和 Non-Scannable Cell，示例如下：

```
rpt_scan_element -type all
Design: top
--------------------------------------------------------------------------------------------------------
-----
Name       Type       Length   InstanceName    SiSoClocks   ClockEdge  ObjState
--------------------------------------------------------------------------------------------------------
-----
seg1      segment       5       /U1/dff_A1     clk1, clk2     (+,+)
user_defined_nonscannable
                                /U1/dff_A5
seg2      segment       8       /U2/dff_B1     clk3, clk4     (+,+)     inferred_nonscannable
                                /U2/dff_B8
DFFR1      dff          1       /U3             clk5           (-)       DRC_fail
DFFR2      dff          1       /U4             clk6           (+)       scannable
DED_WRP1   ded_wrp_cell 1       /U5            wrp_clk1        (+)       scannable
```

报告的内容说明如下表所示：

表 2-2 Scannable 和 Non-Scannable Cell 报告说明

表头字段	内容说明
Name	Segment 名称或 Functional FF 名称。
Type	Cell 类型，包括 Segment 或 Functional FF 的类型。
Length	Segment 或 Functional FF 的位数。
InstanceName	Segment 或 Functional FF 的层级路径。
SI_SO_clocks	Segment 或 Functional FF 的首尾 Scan 时钟信号。
Clock_edges	Segment 或 Functional FF 的首尾 Scan 时钟极性。
Obj_state	Segment 或 Functional FF 的 Scannable 和 Non-Scannable 属性。

2.1.2.3 识别已有插链信息

通过 CTL 文件识别已有插链信息

CTL（Core Test Language）文件描述了设计中已有的插链结构。用户在读入 CTL 文件并指定设计顶层后，可直接在顶层执行 Scan Insertion 操作。Scan 引擎支持将 Lirabry 中 CTL 描述的 Scan Chain 结构添加到顶层的 Scan Chain 结构中。

读取 CTL 文件的示例如下：

```
load_ctl -module module_name1 ./ctl_files1
```

其中，*module_name1* 为 CTL 加载的设计名称，*ctl_files1* 为 CTL 文件的位置。

说明

- 该命令将 CTL 文件读入至指定 Module 中，支持读入多份 CTL 文件。若 Module 中已有 CTL 信息，则新读取的 CTL 文件将覆盖已存在的 CTL 文件；
- 仅支持将 Module 对象指定为 Liberty 中的 Cell，不支持指定为 Verilog 中的 Module。若设计不存在，Scan 引擎将在执行 **present_design** 命令时报错；
- 支持自动解析 CTL 文件中的 Scan Segment 并使其参与插链；但不支持通过 add_scan_chain 命令直接指定该 Segment；
- 支持通过 **rpt_scan_segment** 命令报出 CTL 文件内的 Scan Chain；
- 支持将插完链的结构通过 **dump_ctl** 命令导出 CTL 文件；
- 不支持 CTL 文件中 pseudo_port/hookup_pin 的解析。如果读入的 CTL 文件中含有 pseudo_port 或 hookup_pin，Scan 引擎将报出对应信息丢失的错误信息；
- 不支持校验 CTL 文件内的 Scan Chain 结构是否正确，即不支持对 Scan Chain SiSo 进行追踪，校验 Scan Chain 结构是否不完整或存在多条链结构；
- 不支持通过命令 **add_scan_partition** 将 CTL 实例化后的所有 Instance 指定为 Partition；
- 不支持通过命令 **set_scan_skew_group** 将 CTL 实例化后的所有 Instance 指定为 Scan Skew Group；
- 不支持通过 **set_scan_chain** 及 **set_scan_segment** 命令将 CTL 实例化后的所有 Instance 指定为已有插链信息；
- 不支持通过 **rpt_scan_chain_cell** 和 **rpt_scan_element** 命令报出 CTL 文件内部的 Scan Cell。

通过命令行识别已有插链信息

- 使用命令行定义 Scan Segment

  用户可通过 **set_scan_segment** 命令定义已有的 Scan Segment，且需指定 Scan Segment 中的以下信号对应的引脚信息：scan_enable、scan_data_in 以及 scan_data_out。

用户可在该命令中使用**-access** 参数，指定信号类型（singal_type）以及信号对应的引脚信息（pin_name）。例如，用户可使用以下命令指定网表中已存在的 Scan Segment：

```
set_scan_segment seg1 -access {scan_enable /U1/SE scan_data_in /U1/SI scan_data_out /U7/SO} -lockup_exists false
```

说明

- **set_scan_segment** 命令中的时序单元插链顺序固定，不支持用户定义顺序；
- 不支持将 Segment 指定为移位寄存器；
- 不支持通过读入 CTL 文件和 **set_scan_segment** 命令重复指定同一个触发器。

用户可用 **rpt_scan_segment** 命令报出当前设计中的 Scan Segment 信息。示例如下：

```
rpt_scan_segment
Design: top
-------------------------------------------------------------------------------------------------------------------------
-----
Name  SegmentProperty    Length  SIPin      SOPin          ChainName                  SISOClocks        ClockEdges
-------------------------------------------------------------------------------------------------------------------------
-----
seg1  user_defined        5      /scan_in1   /scan_out1       chain1                     clk3, clk4          (+,+)
seg2  inferred_from_ctl   5      /ext_si1    /ext_so1 ScanInsert_top_wrapper_input_chain3 wrp_clk1, wrp_clk1  (+,+)
```

报告的内容说明如下表所示：

表 2-3 Scan Segment 信息说明

表头字段	内容说明
Name	Segment 名称。
Segment_property	Segment 的属性，支持以下类型：user_defined 以及 inferred_from_ctl。 • user_defined：表示用户以 **set_scan_segmen**t 命令定义的 Segment； • inferred_from_ctl：表示用户以读入 CTL 文件定义的 Segment。
Length	Segment 的长度。
SI_pin	Segment 数据输入引脚名称。

表头字段	内容说明
SO_pin	Segment 数据输出引脚名称。
Chain_name	Scan Chain 生成后，Segment 所属的 Chain 名。
SI_SI_clocks	Segment 首尾 Cell 的 Scan 时钟名。
Clock_edges	Segment 首尾 Cell 的 Scan 时钟极性。

- 使用命令行定义 Scan Chain

  用户可通过命令 **set_scan_chain** 定义设计中已有的 Scan Chain，同时需声明 Scan Chain 中以下信号的端口：scan_enable、scan_data_in 以及 scan_data_out。Scan 引擎将根据用户指定的信息进行扫描链追踪，并从设计中提取扫描链。例如，my_chain0 是设计中已存在的扫描链，用户可使用以下命令定义该扫描链：

```
set_scan_chain my_chain0 -scan_enable se -scan_data_in my_si1 -scan_data_out my_so1 -class scan
set_scan_chain my_wrp_chain0 -scan_enable wrp_shift -scan_data_in ext_wSI -scan_data_out
ext_wSO -class wrapper
```

  说明

  该命令支持用户指定 Scan Chain 和 Wrapper Chain。

  - 如果 class 设置为 *scan*，参数项-scan_enable 表示 Basic Chain 的 scan_enable 端口；
  - 如果 class 设置为 *wrapper*，参数项-scan_enble 表示 Wrapper Chain 的 shift_enable 端口；
  - 参数项-scan_enable 不支持指定 Wrapper Chain 的 capture_enable 信号。

  用户可用 **get_scan_chain_name** 命令报出当前设计中的 Scan Chain 名称，示例如下：

```
get_obj_name [get_scan_chain_name]
my_chain_0 1 2 3 4 my_wrp_chain0 wrp_i_1 wrp_i_2 wrp_o_1
```

  用户可用 **get_scan_chain_cell** 命令报出当前设计中 Scan Chain 和 Wrapper Chain 上 Cell，示例如下：

```
get_obj_name [get_scan_chain_cell -chain 1]
U1/sub_u2_aa/a1/sff0 U1/sub_u2_aa/a1/sff1 U1/sub_u2_aa/a1/dff2 U1/sub_u2_aa/a1/dff3
U1/sub_u2_aa/a1/sff2
```

2.1.2.4 配置 Scan Chain

定义 Scan Chain 的数量和长度

用户可以通过以下命令定义 Scan Chain 的数量和长度：

```
set_scan_cfg -chain_count 10 -max_length 100
```

当 chain_count 与 max_length 冲突时，优先遵从 chain_count 的定义。这意味着 Chain 的实际长度可能会超出 max_length 定义的值。每条 Chain 上的 Wrapper Cell 数量按照 Balance 规则进行分配，尽可能接近。

用户可以使用 **add_scan_chain** 命令，将指定的 Scan Elements 单独配置成一条新的 Scan Chain。该命令支持通过参数-include_elements/-ordered_elements 指定自定义 Chain 包含的有序/无序元素列表，支持通过参数-clock 和-edge 指定特定时钟和边沿驱动的元素。该命令支持用户设置该条 Scan Chain 的信号端口，包括：Scan 使能端口、Scan 测试数据输入端口以及 Scan 测试数据输出端口。

命令示例如下：

```
add_scan_chain my_add_chain0 -ordered_elements {dff1, dff2, dff3} -scan_enable SE -scan_data_in SI -scan_data_out SO
```

其中，*{dff1, dff2, dff3}*为指定的 Scan Elements；*SE*、*SI*、*SO* 分别为该条 Scan Chain 的 Scan 使能信号，Scan 测试数据输入信号，以及 Scan 测试数据输出信号连接的端口名。

定义时钟域、时钟边沿

用户可以通过以下命令根据 Scan Cell 的时钟信息配置 Scan Chain：

- 将不同时钟域的触发器混插在一条 Scan Chain 上。示例如下：

  ```
  set_scan_cfg -mix_clocks true
  ```

- 将不同时钟沿（Trailing Edge（TE）、Leading Edge（LE））的触发器混插在一条 Scan Chain 上。示例如下：

  ```
  set_scan_cfg -mix_edges true
  ```

- 将不同时钟域下的触发器视为一个时钟域。示例如下：

  ```
  set_scan_skew_grp -include_clocks {clk1 clk2}
  ```

  其中，{*clk1，clk2*}为不同时钟域下的触发器的时钟信号。

定义内部时钟

- 全局配置时钟信号派生内部时钟的策略。命令示例如下：

```
set_scan_cfg -internal_clocks multi
```

- 将由同一时钟派生的内部时钟驱动的触发器混插在一条 Scan Chain 上。命令示例如下：

```
set_scan_cfg -mix_internal_clocks true
```

定义 Lockup Latch

用户可以通过以下方式配置 Scan Chain 在插链时插入的 Lockup Latch。

- 在时钟域边界添加 Lockup Latch。示例如下：

```
set_scan_cfg -add_lockup true
```

- 在 Scan Chain 的链尾插入 Lockup Latch。命令示例如下：

```
set_scan_cfg -insert_terminal_lockup true
```

定义 Retiming Register

用户可根据需求，通过以下方式配置 Scan Chain 的 Retiming Register。

- 在负沿链首、负沿链尾插入正沿 Retiming Register（DFF 型），采用 set_scan_cfg 命令。示例如下：

```
set_scan_cfg -add_test_retiming_flops begin_and_end
```

- 在链首、链尾插入符合条件的 Retiming Register（SDFF 型）时，采用 set_scan_chain_retiming 命令。示例如下：

```
set_scan_chain_retiming
```

如果用户不设置任何参数项并执行 set_scan_chain_retiming 命令，Scan 引擎会采取以下默认策略：自动为识别和插入的所有扫描链添加 Retiming Register，并确保链首为上升沿触发单元，链尾为下降沿触发单元。仅当原扫描链首尾不满足时钟沿条件时才会添加，符合条件则保留原链。Retiming Register 的时钟、测试使能信号以及层级位置和与相邻最近的单元保持一致；其时钟沿则与相邻最近的单元相反。

Scan 引擎支持用户添加自定义的 Retiming SFF。用户可通过以下命令控制 Retiming Register 的时钟、时钟沿、测试使能信号以及层级位置。示例如下：

```
set_scan_chain_retiming -se SE_glb -location ua/u1 -si_edge falling -si_clk CLK_01 \
                        -so_edge rising -so_clk CLK_10
```

参数说明：

- **se**：指定 Retiming Register 的测试使能信号；
- **location**：指定 Retiming Register 的层级位置；
- **si_clk**：指定链首 Retiming Register 的时钟；

- **so_clk**：指定链尾 Retiming Register 的时钟；
- **si_edge**：指定链首 Retiming Register 的时钟沿；
- **so_edge**：指定链尾 Retiming Register 的时钟沿。

说明

参数-si_clk 和-so_clk 支持适配通过 set_scan_signa 命令的-type clock 参数项指定的源时钟，以及通过-associated_internal_clocks 参数项指定的内部时钟。

定义 Scan Chain 测试数据的端口格式

Scan 引擎支持用户自定义 Scan Chain 的 Scan 测试数据输入端口（scan_data_in）及 Scan 测试数据输出端口（scan_data_out）的名称格式。用户可在自定义的端口名称后面添加以下信息：%d 或者\[%d\]。

- %d：表示 Chain Index。如果添加该信息，Scan 引擎将创建对应的 Single Port。
- \[%d\]：如果添加该信息，Scan 引擎将创建 Bus Port。

其中，Single Port 的名称格式同时对当前顶层设计和子设计中由于插链而生成的 Scan 测试数据输入和输出端口生效；Bus Port 名称格式仅对当前顶层设计生效，对子设计不生效。

由 Scan 引擎生成的 Scan Chain 的 scan_data_in 和 scan_data_out 信号，默认格式分别为 test_si%d 和 test_so%d。用户可通过以下命令，自定义 Scan Chain 测试数据端口的名称：

```
set_scan_cfg -si_port_format my_si%d -so_port_format my_so%d
```

报告 Scan 配置

Scan 引擎支持报告出用户定义的 Scan 配置。示例如下：

```
rpt_scan_cfg
------------------------------------------------------------------------
ScanConfigurationParameter      Value
------------------------------------------------------------------------
chain_count                             200
max_length                              300
mix_edges                               False
mix_clocks                              False
mix_internal_clocks                     False
add_lockup                              True
add_test_retiming_flops                 Begin_and_end
insert_terminal_lockup                  False
replace                                 True
respect_sff_se_connection               False
si_port_format                          test_si%d
```

```
so_port_format                        test_so%d
-------------------------------------------------------------------------
```

删除 Scan Chain

用户可以使用 remove_scan_chain 命令删除定义的 Scan Chain。示例如下：

```
set_scan_chain my_chain_0 -class scan -scan_enable SE -scan_data_in SI1 -scan_data_out SO1
add_scan_chain my_chain_1 -clock clk0_0
remove_scan_chain -chain {my_chain_0 my_chain_1}
```

其中 *my_chain_0* 和 *my_chain_1* 分别是被 set_scan_chain 和 add_scan_chain 指定成功的 Scan Chain Name。

说明

只有由 set_scan_chain 或 add_scan_chain 命令成功创建的扫描链，才可被 set_scan_chain 命令删除。

Scan Chain 示例

如下展示了一个 TCL 脚本，它执行了一个基本的 Scan Chain 配置及插链流程，脚本中标识了每一个步骤的含义。

```
Example script
# load CTL test model and library
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib

# load the synthesized netlist
load_netlist ./inputs/pre_scan_demo.v

# Set current working design
present_design demo_top

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK}
set_scan_signal -type scan_enable -off_state 0 -port SCAN_EN

# Configure scan chain
set_scan_cfg -max_length 250 -mix_edges true -mix_clocks true
set_scan_chain my_core_chain1 -scan_enable SCAN_EN \
-scan_data_in SI1 -scan_data_out SO1 -class scan

# DRC
examine_scan_drc

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic
```

```
# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl "./post_scan_demo.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/post_scan_demo.v"
```

2.1.2.5 配置 Clock-Gating

在插链中配置 Clock-Gating 连线

用户可以通过如下方式指定 scan_enable 信号控制的 Clock-Gating 实例。示例如下：

```
set_dft_clock_gating_cfg -exclude_elements {icg5 icg6}
```

其中，通过**-exclude_elements** 添加的 Clock-Gating 实例会保留原有的控制信号，设计中其他满足连接条件的 Clock-Gating 实例将由 scan_enable 信号控制。

set_dft_clock_gating_cfg 命令的作用范围包含 Hierarchy Cell 及 Leaf Cell。

Scan 引擎在执行 **insert_dft_logic -stitch** 命令时，可被 scan_enable 信号控制的 Clock-Gating 实例应该满足以下条件：

1. 该 Clock-Gating 没有 dont_touch 属性；
2. Clock-Gating 的 TE 端为 Tie Off 状态。Tie Off 状态分为以下情况：
   - Clock-Gating 的 TE 端仅有一个 fan-in 信号源，且为 Tie Off 状态；
   - Clock-Gating 的 TE 端有多个 fan-in 信号源，均为 Tie 状态，它们的逻辑运算结果使得 ICG TE 端为 Tie Off 状态。

Scan 引擎在执行 **insert_dft_logic -connect_icg_only** 命令时，会根据全局变量 **icg_auto_identify** 的取值，决定 Clock-Gating 连接的范围。

- 当 **icg_auto_identify** 设置为 **false** 时，Scan 引擎不会识别设计中的功能 Clock-Gating 实例。此时引擎在执行 **insert_dft_logic -connect_icg_only** 命令时，只考虑综合阶段插入的、具有 insert_icg 属性的 Clock-Gating 实例。引擎会结合 **set_dft_clock_gating_cfg** 命令的设定，将具有 insert_icg 属性的 Clock-Gating 实例的 TE 端连接至 scan_enable 信号；
- 当 **icg_auto_identify** 设置为 **true** 时，Scan 引擎会识别设计中的功能 Clock-Gating 实例。此时引擎在执行 **insert_dft_logic -connect_icg_only** 命令时，会考虑综合阶段插入的、具有 insert_icg 属性的 Clock-Gating 实例以及设计中的功能 Clock-Gating 实例。引擎会结合 **set_dft_clock_gating_cfg** 命令的设定，将对应 Clock-Gating 实例的 TE 端连接至 scan_enable 信号。

2.1.2.6 配置 Wrapper Chain

Wrapper 技术是一种分层次测试的技术。该技术在设计分区的边界配置 Wrapper Chain，在设计分区的核心配置 Scan Chain。Wrapper Chain 的存在提高了从外部对输入和输出信号的控制和观测能力。Scan Chain 的存在使工程师更加容易地访问和控制设计内部。Wrapper 技术提高了测试的可控性和可观测性。

使用 Wrapper 技术完成插链后得到的设计可用于 EXTEST Mode。EXTEST Mode 是一种专用于测试核心（Core）外部逻辑电路的技术。在 EXTEST Mode 下，使用 Output Wrapper Chain 发送测试向量，Input Wrapper Chain 捕获数据。该模式能够在不依赖核心内部逻辑的情况下，验证核心外围的互连和外部逻辑的正确性。

定义 Wrapper Chain 的数量和长度

用户可使用以下命令配置 Wrapper Chain 的数量和长度：

```
set_wrapper_cfg -chain_count 3 -chain_length 500
```

当 chain_length 与 chain_count 冲突时，优先遵从 chain_count 的定义，这意味着 Chain 的实际长度可能会超出 max_length 定义的值。每条 Chain 上的 Wrapper Cell 数量按照 Balance 规则进行分配，尽可能接近。该命令仅支持指定 Wrapper Chain 的数量和长度。

定义 Wrapper Chain 的内部时钟混合策略

用户可使用以下命令将由同一时钟派生的内部时钟驱动的触发器混插到一条 Wrapper Chain 上：

```
set_wrapper_cfg -mix_internal_clocks true
```

定义 Wrapper 策略

- 指定全局端口的 Wrapper 插链策略

  用户可使用以下命令将 Wrapper 插链策略定义为对可用端口直接插入 Dedicated Wrapper Cell：

  ```
  set_wrapper_cfg -style dedicated
  ```

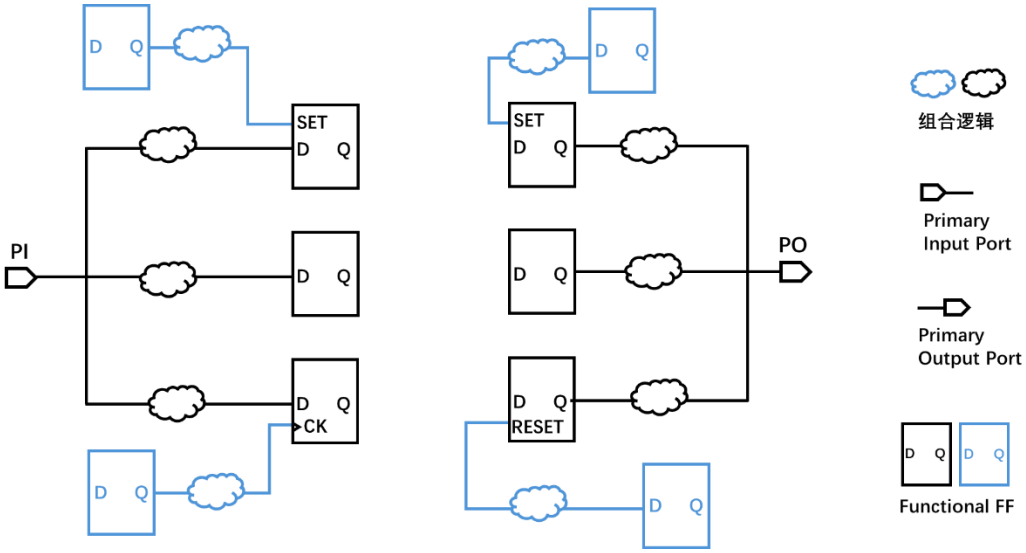
  用户可使用以下命令将 Wrapper 插链策略定义为最大程度上复用 Functional Cell 作为 Wrapper Cell：

  ```
  set_wrapper_cfg -style shared
  ```

  当开启最大程度复用模式时，Scan 引擎会结合**-reuse_threshold** 和**-depth_threshold** 参数项的值，尽可能地将设计中的 Functional FF 复用为 Shared Wrapper Cell。

下图展示了 Wrapper-Port 追踪路径拓扑图。

图 2-2 Wrapper-Port 追踪路径拓扑图



Scan 引擎开启最大程度复用模式后，会对 Primary Input（PI）端口和 Primary Output（PO）端口进行 Direct 追踪以及 Feedback 追踪。下文以 PI 端口为例，介绍两种追踪模式：

- Direct 追踪

  从 PI 端口向后，进行信号传播分析。穿透可能存在的组合逻辑电路，将路径上遇到的首个 Functional FF 标记为目标单元。该单元需满足所有控制端口（Clock、Data、Set 和 Reset）可控的要求。

- Feedback 追踪

  从 Direct 追踪到的目标单元的控制端口（Clock、Data、Set 和 Reset）向前，进行信号传播分析。穿透可能存在的组合逻辑电路，将路径上遇到的首个 Functional FF 标记为目标单元。

接着，Scan 引擎会根据追踪的结果以及追踪端口的类型，采用不同的复用策略：

- PI 端口
  - 如果 Direct 追踪和 Feedback 追踪的目标单元的总数超过参数**-reuse_threshold** 规定的阈值，则需要对该 PI 端口插入 Dedicated Input Wrapper Cell；
  - 对于 Direct 追踪和 Feedback 追踪到的目标单元，需考虑每一条追踪路径上的组合逻辑深度。如果任意一条追踪路径上的组合逻辑深度超过参数**-depth_threshold** 规定的阈值，则需要对该 PI 端口插入 Dedicated Input Wrapper Cell；

- 如果 Direct 追踪和 Feedback 追踪的目标单元的总数不超过参数**-reuse_threshold** 规定的阈值且每一条追踪路径上的组合逻辑深度均不超过参数**-depth_threshold** 规定的阈值，则 Direct 追踪到的目标单元作为 Shared Input Wrapper Cell，Feedback 追踪到的目标单元作为 Shared Output Wrapper Cell。

- PO 端口
  - 如果 Direct 追踪和 Feedback 追踪的目标单元的总数超过参数-reuse_threshold 规定的阈值，则需要对该 PO 端口插入 Dedicated Output Wrapper Cell；
  - 对于 Direct 追踪和 Feedback 追踪到的目标单元，需考虑每一条追踪路径上的组合逻辑深度。如果任意一条追踪路径上的组合逻辑深度超过参数-depth_threshold 规定的阈值，则需要对该 PO 端口插入 Dedicated Output Wrapper Cell；
  - 如果 Direct 追踪和 Feedback 追踪的目标单元的总数不超过参数-reuse_threshold 规定的阈值且每一条追踪路径上的组合逻辑深度均不超过参数-depth_threshold 规定的阈值，则 Direct 追踪和 Feedback 追踪到的目标单元均作为 Shared Output Wrapper Cell。

- 指定特定端口的 Wrapper 插链策略

  用户可通过以下命令指定端口的 Wrapper 策略是直接插入 Dedicated Wrapper Cell，并且为指定端口设置 Dedicated Wrapper Cell 的类型：

  ```
  set_wrapper_cfg -style dedicated -port {port1 port2} -dedicated_cell_type user_defined
  ```

  其中，*{port1 port2}*为指定待插入 Dedicated Wrapper Cell 的端口；*user_defined* 表示 Dedicated Wrapper Cell 的类型为用户自定义。

  用户可通过以下命令指定端口的 Wrapper 策略是最大程度上复用 Functional Cell 作为 Wrapper Cell：

  ```
  set_wrapper_cfg -style shared -port {port3 port4}
  ```

  其中，*{port3 port4}*为指定 Wrapper 策略是最大程度上复用 Functional Cell 作为 Wrapper Cell 的端口。

  用户可通过以下命令指定不插入 Wrapper Cell 的端口：

  ```
  set_wrapper_cfg -style none -port {port5 port6}
  ```

  其中，*{port5 port6}*为不插入 Wrapper Cell 的端口。

  用户可通过以下命令消除指定端口的 wrapper 策略：

  ```
  remove_wrapper_cfg -port {port1 port3 port6}
  ```

其中，*port1*、*port3* 和 *port6* 是被 set_wrapper_cfg -port 指定成功的 Port Name（必须条件）。

- 指定特定 Cell 不参与 Wrapper 插链

  用户可通过以下命令指定特定 Cell 不参与 Wrapper 插链：

  ```
  set_wrapper_cfg -exclude {ua/sff0 ua/sff1}
  ```

  其中，*{ua/sff0 ua/sff1}*为不参与 Wrapper 插链的 Cell。

  用户可通过以下命令消除指定 Cell 的 wrapper 策略：

  ```
  remove_wrapper_cfg -exclude {ua/sff0}
  ```

  其中，*ua/sff0* 是被 set_wrapper_cfg -exclude 指定成功的 Cell Name（必须条件）。

- 控制 Input Wrapper Cell 和 Output Wrapper Cell 混合插入策略

  用户可通过以下命令，指定是否允许将 Input Wrapper Cell 和 Output Wrapper Cell 混合插入到同一条 Wrapper Chain 中：

  ```
  set_wrapper_cfg -mix_cell true
  ```

  其中，参数项**-mix_cell** 的值为 **true** 时，允许混合插入，即 Input 和 Output Wrapper Cell 可共存于同一条 Wrapper Chain；该参数项的值为 **false** 时，不允许混合插入，Input 和 Output Wrapper Cell 将分别参与到独立的 Wrapper Chain。

- Scan 引擎自动识别插链策略

  下表介绍了为不同类型的 Primary 端口插入 Wrapper Cell 的策略：

  表 2-6 不同类型 Primary Port 的 Wrapper 策略

Port 类型	**是否插 Wrapper Cell**
normal in/out data port	是
clock/set/reset port	否
Bidi port	是
Tri-state port	是
DFT signal port(SE/SI/SO)	否

  如果 Scan 引擎识别到不满足插入 Shared Wrapper Cell 的端口，会使用 Dedicated Wrapper Cell 进行插入或者不插入 Wrapper Cell。下表介绍了不同场景下的 Wrapper 插链策略：

表 2-7 不同场景下的 Wrapper 策略

场景描述	Wrapper 策略
组合逻辑深度超过 depth_threshold	该端口使用 Dedicated Wrapper Cell
PI/PO 连接的时序单元属于 Pre-Existing Scan Chain	
复用的 Wrapper Cell 超过 reuse_threshold	
Trace 遇到 ICG 的所有输入引脚（包括 enable、scan_enable、clock）	
Trace 到 PO/PI 的情况下，除了 buffer/inverter 外，没有 Trace 到任何时序元件或组合逻辑	该端口不插 Wrapper Cell
Trace 遇到 Inferred Dedicated Wrapper Cell	
PI/PO 为 Unconnected 端口	
Trace 遇到 Blackbox	该条 Path 不继续 Trace
Trace 遇到 dont_touch 属性 DFF	
Feedback Trace 时遇到 ICG	
Trace 遇到被 set_wrapper_cfg -exclude 指定的 Cell	
Trace 遇到 Non-Scannable Cell	根据对应参数项的取值选择 Wrapper 策略
Trace 遇到 Multi-Beat Cell	
Trace 遇到 three-state port	在数据输出和控制路径上插入 Wrapper Cell
Trace 遇到 bidirectional port	在数据输出、控制路径以及数据输入路径上均插入 Wrapper Cell

说明

- Scan 引擎仅支持对当前设计做 Wrapper 插链，不支持对子模块设计单独做 Wrapper 插链；

- 如果数据端口只连接单个有 dont_touch 属性的 DFF，该端口不插 Wrapper。如果数据端口连接多个 Cell，其中包括 dont_touch 属性的 DFF，在该路径上不继续分析且不插 Wrapper，其他路径正常分析 Wrapper；
- 有 dont_touch 属性的 DFF 不计入 reuse_threshold；
- 有 dont_touch 属性的 DFF 若处于移位寄存器中，则可忽略 dont_touch 属性。该 DFF 可替换为 Shared Wrapper Cell。

- 指定端口追踪到 Non-Scannable Cell 时的 Wrapper 策略

  用户可通过以下命令，配置当 Wrapper 分析中端口追踪到 Non-Scannable Cell 时所采用的策略：

  ```
  set_wrapper_cfg -add_dedicated_cell_for_nonscan false
  ```

  其中，参数项**-add_dedicated_cell_for_nonscan** 的值被设置为 **true** 时，Scan 引擎在做 Wrapper 分析时，会对追踪到 Non-Scannable Cell 的端口插入 Dedicated Wrapper Cell；该参数项被设置为 **false** 时，做 Wrapper 的数据端口将忽略追踪到 Non-Scannable Cell 的路径，存在 Non-scannable Cell 的路径将不会成为该数据端口插入 Dedicated Wrapper Cell 的依据。

- 指定端口追踪到 Multi-Beat Cell 时的 Wrapper 策略

  用户可通过以下命令，配置当 Wrapper 分析中端口追踪到 Multi-Beat Cell 时所采用的策略：

  ```
  set_wrapper_cfg -add_dedicated_cell_for_multibeat true
  ```

  其中，参数项**-add_dedicated_cell_for_multibeat** 的值被设置为 **true** 时，Scan 引擎会在追踪到 Multi-Beat Cell 的端口上插入 Dedicated Wrapper Cell；当该参数项被设置为 **false** 时，对追踪到 Multi-Beat Cell 的端口，Multi-Beat Cell 将会作为 Shared Wrapper Cell 加入 Wrapper Chain。

- 指定是否允许插入 Dedicated Wrapper Cell

  用户可通过以下命令完成操作：

  ```
  set_wrapper_cfg -no_dedicated_wrapper_cells enable
  ```

  该参数控制 Scan 引擎是否在目标位置插入 **Dedicated Wrapper Cell**：设为 **enable** 则不插入，设为 **disable** 则执行插入。

定义 Wrapper Cell 类型

Wrapper Cell 包括 Dedicated 和 Shared 两类：

- Dedicated Wrapper Cell

  Scan 引擎目前支持内建的 WC_D1 类型及用户自定义的 Dedicated Wrapper Cell。用户自定义的 Dedicated Wrapper Cell 端口列表对应以下信号：{shift_clk, capture_en, shift_en, cti, cto, cfi, cfo}。

例如，用户自定义的 Dedicated Wrapper Cell 的 Module 名为 MY_INPUT_WRAPPER_CELL，其 Module 端口列表为：{subsys_clk, capture_dr, shift_dr, si, so, data_in, data_out}。

那么可以通过定义方式添加该 Module 为 Dedicated Wrapper Cell：

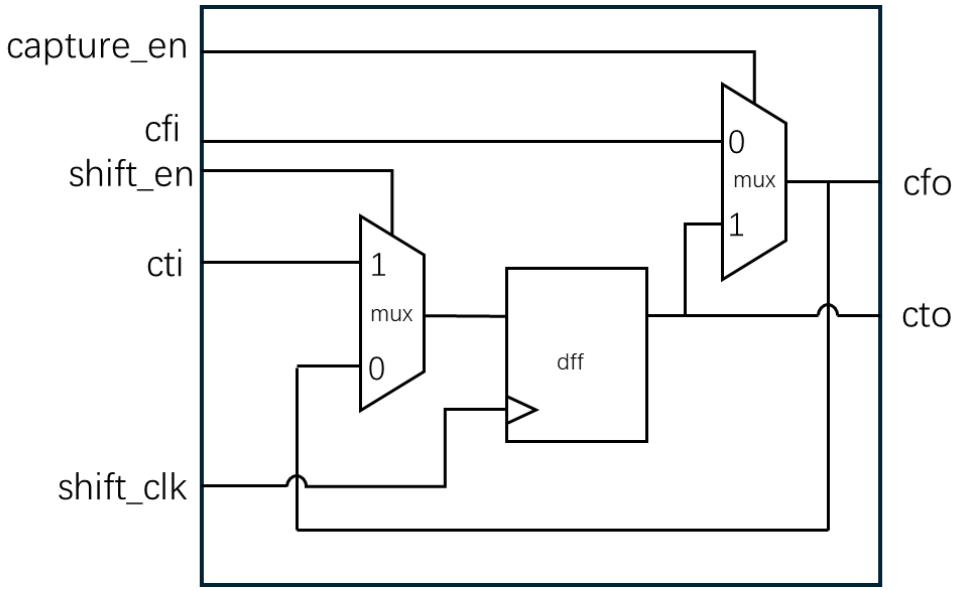
```
add_dedicated_wrapper_cell_type -design_name MY_INPUT_WRP_CELL  \
-interface {shift_clk subsys_clk h capture_en capture_dr h shift_en shift_dr h \
cti si h cto so h cfi data_in h cfo data_out h}
```

对于用户添加的 Dedicated Wrapper Cell，可以通过如下方式报出：

```
rpt_dedicated_wrapper_cell_type
Design: demo_top
------------------------------------------------------------------------------------------------------
DesignName  ShiftClock    CaptureEnable        ShiftEnable     CoreTestInput    CoreTestOutput
          CoreFunctionInput          CoreFunctionOutput
------------------------------------------------------------------------------------------------------
MY_INPUT_WRP_CELL subsys_clk(h) capture_dr(h) shift_dr(h) si(h) so(h) data_in(h) data_out(h)
------------------------------------------------------------------------------------------------------
```

Scan 引擎内建的 WC_D1 类型电路结构如下图所示：

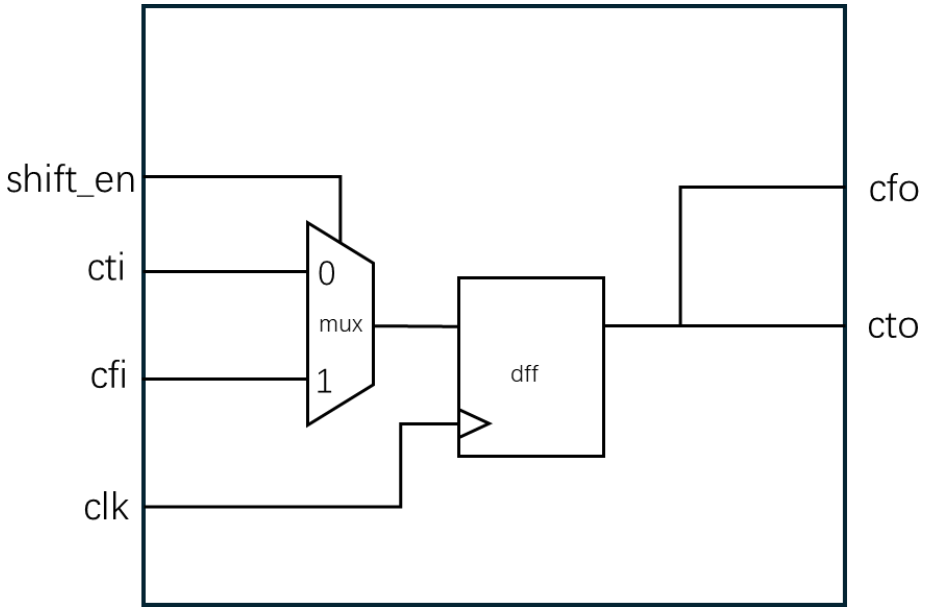
图 2-3 内建 Dedicated Wrapper Cell WC_D1 类型电路结构



- Shared Wrapper Cell

  Scan 引擎目前仅支持 Shifting 类型的 Shared Wrapper Cell，其电路结构如下图所示：

  图 2-4 内建 Shared Wrapper Cell WC_S1 类型电路结构



Scan 引擎将 Scannable Existing Scan Cell 复用为 Shared Wrapper Cell。DFF 的 D 端插入一个 Mux，两个输入端分别接 cfi 和 cti，控制端为 shift_en。DFF 的 Q 端既是 cfo 又是 cto。

仅当 shift_en 使能时，测试数据通过 cti 移入，否则始终处于 Function 模式下。

定义 Dedicated Wrapper Cell 的时钟

Scan 引擎支持为 Dedicated Wrapper Cell 设置 Dedicated Wrapper 时钟，为 Shared Wrapper Cell 复用 Function 时钟。用户可用以下命令定义 Dedicated Wrapper 时钟：

```
set_scan_signal -type wrapper_clock -port wrp_clk
```

用户可以为 Dedicated Wrapper Cell 显式指定使用的时钟，如没有定义则使用工具生成的 Dedicated Wrapper 时钟。

定义 Wrapper Chain 的 Wrapper Shift 信号

Scan 引擎为 Input 和 Output Wrapper Chain 分别生成 Wrapper Shift 信号，并分以下场景：

- 若分析 Wrapper Cells 的结果中存在 Dedicated Wrapper Cell，则生成以下 Wrapper Shift 信号：
  - input_shift_enable：连接 Input Wrapper Cell（Dedicated/Shared）的 shift_en 引脚；
  - output_shift_enable：连接 Output Wrapper Cell（Dedicated/Shared）的 shift_en 引脚；

- input_capture_enable：连接 Input Wrapper Cell（仅 Dedicated）的 capture_en 引脚；
- output_capture_enable：连接 Output Wrapper Cell（仅 Dedicated）的 capture_en 引脚。
- 若分析 Wrapper Cells 的结果中只有 Shared Wrapper Cell，则生成以下 Wrapper Shift 信号：
- input_shift_enable：连接 Input Shared Wrapper Cell 的 shift_en 引脚；
- output_shift_enable：连接 Output Shared Wrapper Cell 的 shift_en 引脚。

用户可使用以下命令自定义 Wrapper Shift 信号的端口名：

```
set_wrapper_cfg -input_shift_enable port_name1 -output_shift_enable port_name2 \
-input_capture_enable port_name3 -output_capture_enable port_name4
```

如果端口名对应的端口已存在且类型（in/out）正确，则使用这个端口；如果该端口名称对应的端口已存在但类型错误或端口不存在，则报错。若用户不指定，Scan 引擎会自动创建新端口作为 Wrapper 相关的测试使能信号端口。

为 Dedicated Wrapper Cells 创建子模块

用户可使用以下命令为 Dedicated Wrapper Cells 创建子模块：

```
set_wrapper_cfg -add_dedicated_cells_module true
```

其中，当参数项**-add_dedicated_cells_module** 设置为 **true** 时，Scan 引擎会将所有 Dedicated Cell 放到同一个子模块下。当该参数设置为 **false** 时，Scan 引擎不会为 Dedicated Wrapper Cells 专门创建子模块。

报告 Wrapper 配置

Scan 引擎支持报告出用户定义的 Wrapper 配置，包括全局的 Wrapper 配置和针对端口的 Wrapper 配置。示例如下：

```
rpt_wrapper_cfg
Design: demo_top
------------------------------------------------------------------
WrapperConfigurationParameter    Value
------------------------------------------------------------------
enable                           Y
chain_count                      500
max_length                       undefined
mix_internal_clocks              true
style                            shared
dedicated_cell_type              WC_D1
shared_cell_type                 WC_S1
input_shift_enable               wrp_in_sft_en
```

```
output_shift_enable                  wrp_out_sft_en
input_capture_enable                 wrp_in_cap_en
output_capture_enable                wrp_out_cap_en
si_port_format                       wrp_si%d
so_port_format                       wrp_so%d
reuse_threshold                      10
depth_threshold                      20
add_dedicated_cell_for_nonscan       true
add_dedicated_cell_for_multibeat     true
add_dedicated_cells_module           true
mix_cell                             false
no_dedicated_wrapper_cells_          disable
--------------------------------------------------------------------

rpt_wrapper_implementation
rpt_wrapper_implementation
Design: demo_top
--------------------------------------------------------------------------------------------------------------------
---------------------------------
No.  PortName(I/O) WrappingStyle UnWrapperReason DedictedReason DWCType SWCType WrapperInstance
ReuseThreshold DepthThreshold
--------------------------------------------------------------------------------------------------------------------
---------------------------------
0    data_in_1       dedicated       --           user_defined user_defined     --
top/in1_wrp_inst 32              32
1    data_in_2       dedicated       --           user_defined user_defined     --
top/in2_wrp_inst 32              32
2     data_out_1     shared          --               --        WC_D1           --
top_sff1         10              20
3     data_out_2     shared          --               --        WC_D1           --
top_sff2         10              20
4     data_in_n      none       user_defined          --         --             --
--               --             --
--------------------------------------------------------------------------------------------------------------------
---------------------------------
rpt_wrapper_implementation -port {data_in_1 data_in_2}
Design: demo_top
--------------------------------------------------------------------------------------------------------------------
---------------------------------
No.  PortName(I/O) WrappingStyle UnWrapperReason DedictedReason DWCType SWCType WrapperInstance
ReuseThreshold DepthThreshold
--------------------------------------------------------------------------------------------------------------------
---------------------------------
0    data_in_1       dedicated       --           user_defined user_defined -- top/in1_wrp_inst
32              32
1    data_in_2       dedicated       --           user_defined user_defined -- top/in2_wrp_inst
32              32
--------------------------------------------------------------------------------------------------------------------
---------------------------------
# load the synthesized netlist
load_netlist ./inputs/pre_scan_demo.v
```

Wrapper Chain 示例

如下展示了一个 TCL 脚本。用户可使用该脚本执行一个基本的 Wrapper Chain 配置及插链流程，脚本中标识了每一个步骤的含义。

```
Example script
# load CTL test model and library
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib

# load the synthesized netlist
load_netlist ./inputs/pre_scan_demo.v

# Set current working design
present_design demo_top

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK}
set_scan_signal -type wrapper_clock -port WRCK
set_scan_signal -type scan_enable -off_state 0 -port SCAN_EN

# Configure wrapper
set_wrapper_cfg enable
set_wrapper_cfg -chain_count 2 -style shared -reuse_threshold 16
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}
add_dedicated_wrapper_cell_type -design_name MY_INPUT_WRAPPER_CELL -interface \
{shift_clk subsys_clk h capture_en capture_dr h shift_en shift_dr h cti si h cto so \
h cfi data_in h cfo data_out h}
rpt_wrapper_cfg
rpt_wrapper_implementation

# Configure scan
set_current_scan_partition Default_Partition
set_scan_cfg -max_length 300

# DRC
examine_scan_drc
rpt_scan_signal -type clock

# Analyze and preview the different scan chains before stitching the chainsexamine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic
rpt_scan_chain -class all
rpt_scan_element -type nonscannable

# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl "./post_scan_demo.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/post_scan_demo.v"
```

2.1.2.7 配置 DFT Partition

用户可以通过 **add_scan_partition** 命令添加 Partition，并为 Partition 添加 Cell。

Scan 引擎支持通过以下方式管理 Partition 中的 Cell：

- 显式指定 Partition 中包含的 Cell
- 显式指定 Partition 中需要排除的 Cell
- 通过指定时钟或者时钟边沿，由 Scan 引擎追踪并添加对应的 Cell

用户通过不同方式指定 Partition 内的 Cell 的示例如下：

```
add_scan_partition p1 -include core1/subA
add_scan_partition p2 -include { core1/subB core1/subC } -exclude { core1/subB/u1 }
add_scan_partition p3 -clocks clk1 -rising_edge_clocks clk2
```

Scan 引擎支持将指定时钟域中的 Functional FF 添加到 Partition 中。例如用户可通过以下命令，将 clk1 和 clk2 驱动的 Functional FF 添加到 Partition part2 中：

```
add_scan_partition part2 -clocks {clk1 clk2}
```

说明

当用户以时钟域的形式定义 Partition 时，仅支持指定源时钟，不支持指定内部时钟。如果指定的时钟端口被定义了 Associated Internal Clock，则 Partition 包含 Associated Internal Clock 对应的 Functional FF。

此外，用户可以通过参数项-force，来设置 Partition 的生效顺序。规则如下：默认情况下（无-force），对于一个 Instance，若先后通过多个 add_scan_partition 命令设置重复添加该 Instance，该 Instance 归属于最先设置包含它的 Partition。若用参数项-force，可强行指定该 Instance 的归属的 Partition。

示例如下：

```
add_scan_partition p1 -include A
add_scan_partition p2 -include A/a1 -force
```

-force 强行设置。结果：A/a1 属于 p2，A 中其他非 a1 逻辑属于 p1。

指定当前的 DFT Partition

用户可以通过 **set_current_partition** 命令将某个 Partition 指定为当前的 Partition，包括由用户先前定义的所有 Partition 以及设计中的默认 Partition（Default_Partition）。

用户指定不同 Partition 为当前 Partition 的示例如下：

```
set_current_scan_partition part1
set_current_scan_partition Default_Partition
```

报告 DFT Partition 下的 Cell 信息

用户可以通过 **rpt_scan_partition** 命令报告出所有 Partition 或特定 Partition 相关的信息。

如果在 DRC 前使用这一命令，Scan 引擎仅能报告用户对 Partition 进行的配置；如果在 DRC 后使用，Scan 引擎将报告根据用户的 Partition 配置，由 Scan 引擎追踪到包含在 Partition 下的所有 Cell 信息，以及它们被包含在该 Partition 中的方式。

在 DRC 前使用示例如下：

```
rpt_scan_partition
Design: xxx
-------------------------------------------------------------------------
Partition  Include   Exclude   Clocks    RisingEdgeClocks  FallingEdgeClocks
-------------------------------------------------------------------------
p1                                  clk1
p2                                                            clk2
p3   /core1/subA/cell4
```

报告的内容说明如下表所示：

表 2-8 DFT Partition 信息说明

表头字段	内容说明
partition	DFT Partition 名称。
include	添加到 Partition 内的 Cell。
exclude	排除在 Partition 内的 Cell。
clocks	将指定时钟沿驱动的 Cell 添加到 Partition 内。
rising_edge_clocks	将指定时钟上升沿驱动的 Cell 添加到 Partition 内。
falling_edge_clocks	将指定时钟下降沿驱动的 Cell 添加到 Partition 内。

在 DRC 后使用示例如下：

```
rpt_scan_partition
Design: xxx
-------------------------------------------------------------------------------
Partition           Cell                          ByClock       ByClockEdge
-------------------------------------------------------------------------------
p1                  /core1/subA/cell1               clk1       both
p1                  /core1/subA/cell2               clk1       both
p2                  /core1/subA/cell3               clk2       rising
p3                  /core1/subA/cell4
```

报告的内容说明如下表所示：

表 2-9 Partition 信息说明

表头字段	内容说明
partition	DFT Partition 名称。
cell	经过 DRC 检查后，实际在对应 Partition 内的 Cell。
by_clock	经过 DRC 检查后，实际在对应 Partition 内的 Cell，且先前被用户以 **add_scan_partition -clocks** 的方式定义的 clock 名。
by_clock_edge	经过 DRC 检查后，实际在对应 Partition 内的 Cell，且先前被用户以 **add_scan_partition -rising_edge_clocks** 和**-falling_edge_clocks** 的方式定义的 clock 名。

删除 DFT Partition

用户可使用 remove_scan_partition 命令删除定义的 DFT Partition。示例如下：

```
add_scan_partition p1 -include /core1/subA
remove_scan_partition p1
```

其中 *p1* 是被 add_scan_partition 指定成功的 Partition Name（必须条件）。

配置 DFT Partition 示例

Scan 引擎支持用户基于 Scan Partition 的设置命令进行 Scan Configuration 以及 Wrapper Configuration。具体说明如下：

1. 配置 Partition 内的 Scan Enable 信号。命令示例如下：

   ```
   set_scan_signal -type scan_enable -port SCAN_EN1 -off_state 0 -usage scan
   ```

   在用户自定义的 Partition 中，**set_scan_signal** 命令仅支持通过-usage 参数项指定用于 Scan 连接的 Scan Enable 信号。

2. 配置 Partition 内的 Scan Chain。命令示例如下：

   ```
   set_scan_cfg -max_length 300 -mix_edges true -add_test_retiming_flops begin_and_end
   ```

   在用户自定义的 Partition 中，**set_scan_cfg** 命令支持的参数项包括：-chain_count、-max_length、-mix_edges、-mix_clocks、-add_lockup、-mix_internal_clocks、-add_test_retiming_flops、-respect_sff_se_connection、-replace 和-insert_terminal_lockup。

3. 配置 Partition 内的 Wrapper Chain。命令示例如下：

   ```
   set_wrapper_cfg -chain_count 2 -max_length 100
   ```

在用户自定义的 Partition 中，**set_wrapper_cfg** 命令支持的参数项包括：-chain_count、-max_length、-mix_internal_clocks、-input_shift_enable、-output_shift_enable、-input_capture_enable 和-output_capture_enable。

当用户启用 Wrapper 配置后，Partition 内的 Wrapper Chain 将保持独立性；不同 Partition 内的 Wrapper Cell 不会混合连接。

下例展示了对 Default Partition 及两个用户自定义 Partition 的 Scan 和 Wrapper 配置。

```
Example script
# load CTL test model,library,netlist
# Set current working design
# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK} -off_state 0
set_scan_signal -type wrapper_clock -port WRCK -off_state 0
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0
set_scan_signal -type constant -port CONST1 -constant_value 1

# Configure for default partition
set_current_scan_partition Default_Partition
set_scan_cfg -max_length 300 -add_test_retiming_flops begin_and_end
set_scan_chain  my_core_chain1 -scan_enable SCAN_EN -scan_data_in SI1 -scan_data_out SO1 \
-class scan
set_wrapper_cfg enable -chain_count 10 -style shared -reuse_threshold 16 \
-dedicated_cell_type WC_D1
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}

# Configure partition
add_scan_partition p1 -clocks CLK
add_scan_partition p2 -clocks RXCLK

# Configure scan for partition p1
set_current_scan_partition p1
set_scan_signal -type scan_enable -port SCAN_EN1 -off_state 0 -usage scan
set_scan_cfg -max_length 250 -mix_edges false -mix_clocks true
set_wrapper_cfg -chain_count 2 -max_length 100

# Configure scan for partition p2
set_current_scan_partition p2
set_scan_signal -type scan_enable -port SCAN_EN2 -off_state 0 -usage scan
set_scan_cfg -max_length 200 -mix_edges true -mix_clocks true -insert_terminal_lockup true
set_wrapper_cfg -chain_count 4 -max_length 80

# DRC
examine_scan_drc

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic
```

```
# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl "post_scan_demo.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/post_scan_demo.v"
```

2.1.2.8 配置 IP Group 的 Wrapper Chain

Scan 引擎支持对设计中实例化的 IP 外围插入 Wrapper Chain，提高测试的灵活性。

定义 IP 和 IP Group

用户可使用以下命令，将 Hierachical Instance 定义为 Hard IP：

```
define_dft_ip -type hard -name {Ua Ub Uc Ud}
```

其中，Ua、Ub、Uc 和 Ud 代表设计中的 Hierachical Instance。

目前，参数项**-type** 仅支持将 Hierachical Instance 定义为 Hard IP，暂不支持其他类型的 IP。被指定为 Hard IP 的 Hierachical Instance 必须具有“blackbox”和“ctl_core”属性。

用户可使用以下命令，将一些指定为 IP 的 Hierachical Instances 组合定义成 IP Group：

```
define_dft_ip_group grp1 -ordered_elements {Ua Ub}
```

其中，grp1 是该 IP Group 的名称，Ua 和 Ub 是指定为 Hard IP 的 Hierachical Instances。

配置 IP Group 的 Wrapper Chain

用户可使用以下命令，指定当前的 IP Group：

```
set_current_dft_ip_group grp1
```

当用户将状态切换到特定的 IP Group 后，可以在该 IP Group 下，设定 Wrapper 信号和 Wrapper 配置。

在 IP Group 下，支持通过 **set_scan_signal** 命令配置的用于 Wrapper Chain 的信号类型有：wrapper_clock、wrp_data_in、wrp_data_out、wrp_in_shift_en、wrp_out_shift_en、wrp_in_capture_en 和 wrp_out_capture_en。

在 IP Group 中配置的信号，可通过以下命令报出：

```
rpt_scan_signal -ip_group grp1
```

在 IP Group 下，支持通过 **set_wrapper_cfg** 命令配置该 IP Group 外围插入 Wrapper Chain 的各项策略。

set_wrapper_cfg 命令的各参数项均支持在 IP Group 下配置。其中，参数项**-port** 在顶层设计和 IP Group 中允许指定的范围有所不同。

- 对于顶层设计，**set_wrapper_cfg -port** 允许指定的对象为顶层设计的端口；
- 对于 IP Group，**set_wrapper_cfg -port** 允许指定的对象为 IP 实例化的 Hierachical Instance 的 Pin。

在 IP Group 下，可以通过 **set_wrapper_cfg -port** 对 IP Group 中的 Pin 指定特定的 Wrapper 策略。对于不做 Wrapper 的 Pin，可通过以下命令指定：

```
set_wrapper_cfg -style none -port {Ua/pin1 Ub/pin2}
```

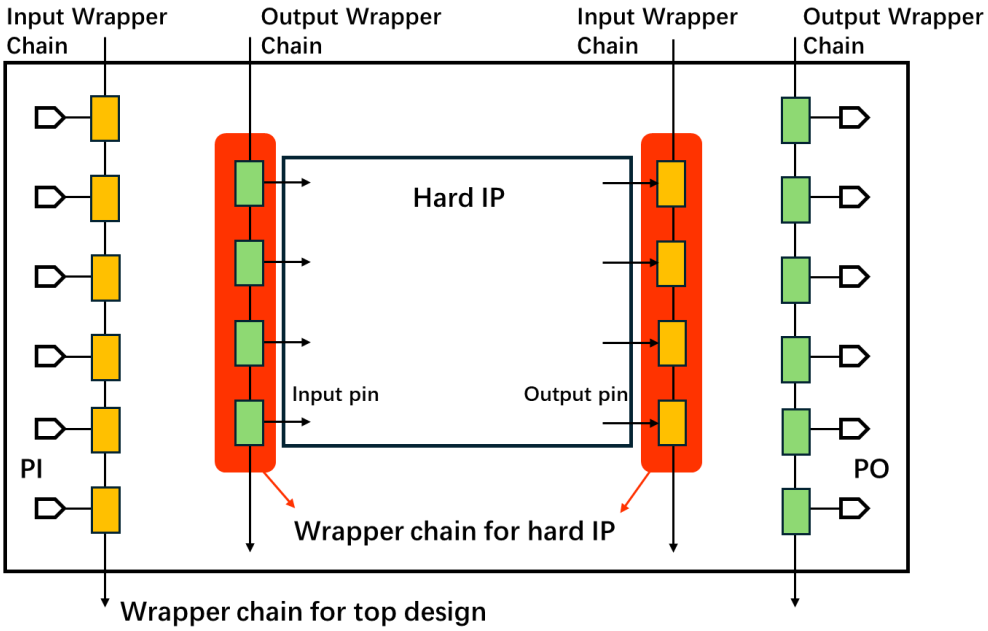
其中，Ua/pin1 和 Ub/pin2 是 IP Group 中的 Pin。

在 IP Group 中配置的 Wrapper Chain 策略，可通过以下命令报出：

```
rpt_wrapper_cfg -ip_group grp1
rpt_wrapper_implementation -ip_group grp1
```

Scan 引擎将会对 IP Group 中的 Hierachical Instances 进行 Wrapper 分析。根据用户设置的 Wrapper 策略，对于需要 Wrapper 控制的 Input Pin，Scan 引擎会对其插入 Output Wrapper Cell；对于需要 Wrapper 控制的 Output Pin，Scan 引擎会对其插入 Input Wrapper Cell。Scan 引擎会根据 **define_dft_ip_group** 命令的**-ordered_elements** 参数项指定的 Hierachical Instances 顺序为这些 Hierachical Instances 对应的 Wrapper Cells 排序。IP Group 的 Wrapper Cell 和顶层设计的 Wrapper Cell 相互独立，不会混合插入到同一条 Wrapper Chain 上。IP Group 外围插入的 Wrapper Chain 如图 2-5 所示。

图 2-5 Hard IP 外围插入 Wrapper Chain 示意图



用户可使用以下命令，跳出 IP Group，将状态切换到顶层设计：

```
set_current_dft_ip_group none
```

用户可使用以下命令，报告 IP Group：

```
rpt_dft_ip_group
Design: test_case
--------------------------------------------------------------------------------------------
WrapperIPGroup       Type            OrderedElements
--------------------------------------------------------------------------------------------
grp1                       hard            Ua Ub
--------------------------------------------------------------------------------------------
```

为 IP Group 插入 Wrapper Chain 示例

如下展示了一个 TCL 脚本。用户可使用该脚本执行一个 IP Group 的 Wrapper Chain 配置及插链流程。脚本中标识了每一个步骤的含义。

```
# Example script

# load the synthesized netlist
load_netlist ./inputs/pre_scan_demo.v

# Set current working design
present_design demo_top

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK}
set_scan_signal -type wrapper_clock -port WRCK
set_scan_signal -type scan_enable -off_state 0 -port SCAN_EN

# Configure wrapper for top design
```

```
set_wrapper_cfg enable
set_wrapper_cfg -chain_count 2 -style shared -reuse_threshold 16
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}
add_dedicated_wrapper_cell_type -design_name MY_INPUT_WRAPPER_CELL -interface \
{shift_clk subsys_clk h capture_en capture_dr h shift_en shift_dr h cti si h cto so \
h cfi data_in h cfo data_out h}
rpt_wrapper_cfg
rpt_wrapper_implementation

# Configure scan
set_current_scan_partition Default_Partition
set_scan_cfg -max_length 300

# Define IP Group
define_dft_ip -type hard -name {Ua Ub Uc Ud}
define_dft_ip_group grp1 -ordered_elements {Ua Ub}

# Define Wrapper signals for IP Group
set_current_dft_ip_group grp1
set_scan_signal -type wrapper_clock -port IP_WRCK
set_scan_signal -type wrp_in_shift_en -off_state 0 -port IP_WRP_IN_EN
set_scan_signal -type wrp_out_shift_en -off_state 0 -port IP_WRP_OUT_EN
rpt_scan_signal -ip_group grp1

# Configure wrapper for IP Group
set_wrapper_cfg enable -max_length 200 -style shared \
     -reuse_threshold 50 -depth_threshold 50 \
     -input_shift_enable IP_WRP_IN_EN -output_shift_enable IP_WRP_OUT_EN
set_wrapper_cfg -style dedicated -port {Ua/in1 Ua/in2 Ub/out1 Ub/out2}
set_wrapper_cfg -style none -port {Ua/pin1 Ub/pin2}
rpt_wrapper_cfg -ip_group grp1
rpt_wrapper_implementation -ip_group grp1

# DRC
examine_scan_drc

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic
rpt_scan_chain -class all
rpt_scan_element -type nonscannable

# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl "./post_scan_demo.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/post_scan_demo.v"
```

2.1.2.9 配置 Scan Replacement

定义 Scan Replacement 配置

用户可使用以下命令，将当前设计中的 DFF 替换为 Scan Data Flip-Flop（SFF）：

```
set_scan_cfg -replace true
insert_dft_logic -replace_only
```

该命令默认对设计中所有的 Scannable DFF 进行替换，而以下几类 Non-Scannable DFF 不会被替换：

- 被 set_scan_element false 命令指定的 DFF；
- 具有 dont_touch 属性且不存在于移位寄存器里的单个 DFF；
- 在 Cell Library 中无对应 SFF 的 DFF；
- 未通过 DRC 检查的 DFF；
- 插链配置为 set_scan_cfg -replace false，且不存在于移位寄存器中的单个 DFF。

Scan 引擎也支持将 SFF 替换成 DFF。需要注意的是：如果 SFF 的 SE pin 处于 tie on 状态或有连接到 cell、port 等功能性连接，则不会被替换成 DFF；仅当 SFF 的 SE pin 处于 tie off 状态且无功能性连接时，用户可使用以下命令，将当前设计中指定的 SFF 或触发 DRC 违例（DFTR1-DFTR3，DFTR9，DFTR17）的 SFF 替换为 DFF：

```
set_scan_element false {Ua/U1/sff0 Ub/U1/sff0 Ua/U1/MB4SFF0 Ua/U1/MB4SFF1:bit0 Ua/U1/MB2SFF0:bit1}
examine_scan_drc
insert_dft_logic -replace_unscan
```

其中，set_scan_element 命令支持指定设计中的单 bit 和多 bit 寄存器的实例化名称。

对于多 bit 寄存器，set_scan_element 命令支持对其中的某些 bit 位进行设置。设置方法是$Multibit_SFF_instance_name:bitX。X 从 0 开始计数，如对于 4bit SFF，X 可取 0、1、2、3。被 set_scan_element false 命令设置指定 bit 位的多 bit 寄存器，将会在执行 insert_dft_logic -replace_unscan 命令时被拆分。被指定的 bit 位被替换成 DFF，其余 bit 位被替换成单 bit SFF。

对于单 bit 和多 bit 寄存器，工具将依据 Liberty 库文件中定义的 SFF 与 DFF 之间的匹配关系进行替换。用户若没有显式指定，Scan 引擎默认以 DFF 和 SFF 的名字，功能及驱动能力匹配；支持用户显式指定，用户可用以下命令显式指定匹配关系：

```
set_scan_cell_mapping $dff_cell $sff_cell
```

其中，*$dff_cell* 代表 Liberty 库文件中的 DFF 名称，*$sff_cell* 代表 Liberty 库文件中的 SFF 名称。

具有 dont_touch 属性且不存在于移位寄存器中的 DFF 不可替换为 SFF；若该 DFF 存在于移位寄存器中，则可忽略 dont_touch 属性。

2.1.3 执行 DRC

DRC（Design Rule Check）指设计规则检查。数字逻辑电路中某些特定的对测试不友好的设计结构会造成电路可测试性下降，根据这些结构的特征可以指定一些设计规则，在 DFT 设计初期，利用这些规则可以识别出逻辑电路中对测试不利的结构。工程师可根据检查结果对电路结构进行优化以提升电路的可测试性。

用户可通过以下命令发起 DRC 检查：

```
examine_scan_drc
```

2.1.3.1 DRC 规则

DRC 检查的规则内容如下表所示：

表 2-10 DRC 检查的规则内容

DRC 规则	描述
DFTR1	检查寄存器时钟是否不可控。(Clock input of DFF is uncontrolled.)
DFTR2	检查寄存器置位是否可控。(Set input of DFF is uncontrolled.)
DFTR3	检查寄存器复位是否可控。(Reset input of DFF is uncontrolled.)
DFTR4	检查 DLAT 时钟是否可控。(Clock input of DLATCH is uncontrolled.)
DFTR5	检查 DLAT 置位是否可控。(Set input of DLATCH is uncontrolled.)
DFTR6	检查 DLAT 复位是否可控。(Reset input of DLATCH is uncontrolled.)
DFTR7	检查 RAM 时钟是否可控。(Clock input of RAM is uncontrolled.)
DFTR8	检查时钟是否能控制任何时序元件。(Clock is unable to capture data when other clocks are set off.)
DFTR9	检查时钟打开时时序元件 Clock Input 是否能正常开启。(Clock input is inactive when clocks are set on.)
DFTR10	检查时钟是否连接到时序元件的 Data Input。(Clock is connected to data input of cell.)
DFTR11	检查时钟是否连接到时序元件的 Clock Input 和 Data Input。(Clock is connected to both clock and data inputs of cell.)

DRC 规则	描述
DFTR12	检查时钟是否连接到时序元件的多个 Clock/Set/Reset Inputs。(Clock is connected to clock/set/reset inputs of cell.)
DFTR13	检查 LS 型的时序元件的 Clock Input 是否捕获到不可靠的数据。(Clock will capture new data on LS input of cell.)
DFTR14	检查 TE 型的时序元件的 Clock Input 是否捕获到不可靠的数据。(Clock will capture new data on TE input of cell.)
DFTR15	检查 LS 型的时序元件的 Clock Input 是否受到新捕获数据的影响。(The path of clock is influenced by the new capturing on LS input of cell.)
DFTR16	检查 TE 型的时序元件的 Clock Input 是否受到新捕获数据的影响。(The path of clock is influenced by the new capturing on TE input of cell.)
DFTR17	检查时序元件的 Clock Input 是否能捕获数据。(Clock input of cell is unable to capture data.)
DFTR-TIE0	检查 Cell 是否有固定值 0。(Cell has constant 0 value.)
DFTR-TIE1	检查 Cell 是否有固定值 1。(Cell has constant 1 value.)
DFTR-L1	检查所有高通 DLAT 是否被 FF 或低通 DLAT 驱动。 (HL is not driven by either a FF or a LL.)
DFTR-L2	检查低通 DLAT 和高通 DLAT 的组合是否可控。 (LL and HL are uncontrolled.)

DRC 规则等级定义如下表所示:

表 2-11 DRC 规则等级定义

规则等级	格式规则	候选级别	说明
Error	Error: xxxx.	Warning、Info	需要解决后才能通过 DRC 检查，也可以通过降级规避。
Warning	Warning: xxxx.	Error、 Info	可能对最终指标（如 Scan Chain 上 FF 的数量等）产生一定负面影响。
Info	Info: xxxx.	无	提供一些可能有用的信息给用户。
Ignore	无	无	仅可作为用户调整后的等级，当规则等级

规则等级	格式规则	候选级别	说明
			设置为 Ignore 时，不会报出该 DRC 规则的任何信息。

DRC 规则等级描述如下表所示：

表 2-12 DRC 规则等级描述

Rule_ID	检查对象	默认规则等级
DFTR1	DFF	Warning
DFTR2	DFF	Warning
DFTR3	DFF	Warning
DFTR4	DLAT	Warning
DFTR5	DLAT	Warning
DFTR6	DLAT	Warning
DFTR7	RAM	Warning
DFTR8	clock port	Warning
DFTR9	DLAT or DFF	Warning
DFTR10	clock port, DLAT or DFF	Warning
DFTR11	clock port, DLAT or DFF	Warning
DFTR12	clock port, DFF	Warning
DFTR13	clock port, DLAT or DFF	Warning
DFTR14	clock port, DLAT or DFF	Warning
DFTR15	clock port, DLAT or DFF	Warning
DFTR16	clock port, DLAT or DFF	Warning
DFTR17	DLAT or DFF	Warning
DFTR-TIE0	DLAT or DFF	Warning
DFTR-TIE1	DLAT or DFF	Warning
DFTR-L1	DLAT	Warning
DFTR-L2	DLAT	Warning

规则编号定义：

规则编号（Rule ID）的命名格式如下：*<RuleClass+Number>*。例如，DFTR1。

违规消息定义：

违规消息（ViolationID）的命名格式如下：*<RuleID-Number>*。其中，*Number* 从 1 开始计数。例如，DFTR1-X，表示规则 DFTR1 的第 X 条违例信息。

2.1.3.2 查询 DRC 信息

用户可通过以下命令报出 DRC 违例结果：

```
rpt_scan_drc_violation
```

报出结果示例如下：

```
//Case-1: 打印违例信息
rpt_scan_drc_violation
------------------------------------------------------
DRC Report
Total violations: 8
------------------------------------------------------
There were 3 DFTR rule DFTR1 fails. (Clock input of DFF is uncontrolled)
There were 2 DFTR rule DFTR3 fails. (Reset input of DFF is uncontrolled)
There were 2 DFTR rule DFTR8 fails. (Clock is unable to capture data when other clocks are set off)
There was 1 DFTR rule DFTR10 fail. (Clock is connected to data input of cell)
1


//Case-2: 打印详细的违例信息
rpt_scan_drc_violation -verbose
------------------------------------------------------
DRC report
Total violations: 8
------------------------------------------------------
Error: Clock input CK of DFF DFF1 is uncontrolled. (DFTR1-1)
Error: Clock input CK of DFF DFF2 is uncontrolled. (DFTR1-2)
Error: Clock input CK of DFF DFF3 is uncontrolled. (DFTR1-3)
Error: Reset input CK of DFF DFF4 is uncontrolled. (DFTR3-1)
Error: Reset input CK of DFF DFF5 is uncontrolled. (DFTR3-2)
Warning: Clock clk1 is unable to capture data when other clocks are set off. (DFTR8-1)
Warning: Clock clk2 is unable to capture data when other clocks are set off. (DFTR8-2)
Warning: Clock clk3 is connected to data input D of DFF DFF1. (DFTR10-1)
1


//Case-3: 打印指定的详细的违例信息
rpt_scan_drc_violation -drc_violation_id {DFTR1 DFTR3-1}
------------------------------------------------------
DRC report
Total violations: 4
------------------------------------------------------
Error: Clock input CK of DFF DFF1 is uncontrolled. (DFTR1-1)
Error: Clock input CK of DFF DFF2 is uncontrolled. (DFTR1-2)
Error: Clock input CK of DFF DFF3 is uncontrolled. (DFTR1-3)
Error: Reset input CK of DFF DFF4 is uncontrolled. (DFTR3-1)
1

//Case-4: 将违例信息输出到指定文件
```

```
rpt_scan_drc_violation >  xx/result/drc_violation_file
```

用户可通过以下命令分析 DRC 违例结果中实例的引脚的信息及引脚上的值：

```
examine_scan_drc_violation -violation_id DFTR1-1
```

报出结果示例如下：

```
//Case-1: 报出 DFTR1-1 违例涉及的单个 Instance，报出信息包括：1. instance_name 及 cell_type 2.
pin_name 及 value 3. trace_path
examine_scan_drc_violation -violation_id DFTR1-1
Error: Clock input CK of DFF DFF1 is uncontrolled. (DFTR1-1)
----------------------------------------
instance_name               cell_type
----------------------------------------
/core1/subA/dff1       DFF

----------------------------------------------------------------
pin         direction         value            connection_pin
----------------------------------------------------------------
"clk"         I               X                /core1/pinA
"d"           I               0                /core1/pinb
"S"           I               X                /core1/pinc
"R"           I               X                /core1/pind
"Q"           O               X                /core1/pine  /core1/pinf
1


//Case-2: 报出 DFTR8-1 违例涉及的 Instance 信息
examine_scan_drc_violation -violation_id DFTR8-1
Warning: Clock clk1 is unable to capture data when other clocks are set off. (DFTR8-1)
1


//Case-3: 报出 DFTR11-1 违例涉及的 Instance 信息
examine_scan_drc_violation  -violation_id DFTR11-1
Warning: Clock portA is connected to both clock and data inputs (/core1/subA/dff2/clk &
/core1/subA/d) of DFF dff2. (DFTR11-1)
----------------------------------------
instance_name                cell_type
----------------------------------------
/core1/subA/dff2               DFF

------------------------------------------------------------
pin          direction        value          connection_pin
------------------------------------------------------------
"clk"         I               X              /core1/pinA
"d"           I               0              /core1/pinb
"S"           I               X              /core1/pinc
"R"           I               X              /core1/pind
"Q"           O               X              /core1/pine  /core1/pinf
1
```

```
//Case-4: 报出 DRC 违例涉及的 Instance 信息 DFTR13 中涉及两个 Instance，报出信息包括这两个
instance_name，cell_type，pin_name 及 value
examine_scan_drc_violation  -violation_id DFTR13-1
Warning: Clock clk will capture new data on LS input d of DLATCH latch1. (DFTR13-1)
Source of violations: input CK of DFF my_dff1.
//violation 中涉及两个 instance，报出信息这两个 instance_name、cell_type、pin_name 及 value

----------------------------------------
instance_name                cell_type
----------------------------------------
/core1/subA/my_latch1     DLATCH

----------------------------------------
pin     direction value     connection_pin
----------------------------------------
"G"      I       X             /core1/pinA
"d"      I       0             /core1/pinb
"Q"      O       X             /core1/pine  /core1/pinf
//pin name 为 G，direction 为 input，DFTR13-1 violation 中 value 为 X

------------------------------------------------
instance_name                cell_type
----------------------------------------
/core1/subA/my_dff1       DFF

------------------------------------------------------
pin     direction    value          connection_pin
------------------------------------------------------
"CK"      I            X             /core2/pinA
"d"       I            0             /core2/pinb
"Q"       O            X             /core2/pine  /core1/pinf
//pin name 为 CK，direction 为 input，DFTR13-1 violation 中 value 为 X
1


//Case-5: 将 DFTR1-1 违例的详细信息输出到指定文件
examine_scan_drc_violation -violation_id DFTR1-1 -file xx/result/drc_DFTR1-1_violation_file
```

2.1.3.3 调整 DRC 违例等级

用户可通过以下命令调整 DRC 违例等级：

```
set_scan_drc_rule_handling {DFTR1 DFTR9  DFTR12} Warning
```

上述示例中，*{DFTR1 DFTR9 DFTR12}*为被调整的 DRC 规则，*Warning* 指调整后的规则等级。该命令表示将 DFTR1、DFTR9、以及 DFTR12 的 DRC 规则等级调整为 Warning。

适用范围：DFTR1-DFTR17、DFTR-TIE0、DFTR-TIE1、DFTR-L1、DFTR-L2。

用户可通过如下方式报告自定义的 DRC 违例等级：

```
rpt_scan_drc_rule_handling
```

```
Design: demo_top
-------------------------------------------------------------------
RuleType    DefaultLevel    SpecifiedLevel    InstRange
-------------------------------------------------------------------
DFTR1           Error           Warning         all
DFTR9           Error           Warning         all
DFTR12         Warning          Warning         all
-------------------------------------------------------------------
```

报告的内容说明如下表所示：

表 2-13 自定义的 DRC 违例等级说明

表头字段	内容说明
Rule_Type	DRC 规则 ID。
Default_Level	默认等级。
Specified_Level	用户指定等级。
Inst_Range	该等级生效的范围。

表 2-14 DRC 违例等级调整

Rule_ID	检查对象	默认的规则等级	允许指定的规则等级
DFTR1	DFF	Warning	Error、Warning
DFTR2	DFF	Warning	Error、Warning
DFTR3	DFF	Warning	Error、Warning
DFTR4	DLAT	Warning	Error、Warning
DFTR5	DLAT	Warning	Error，Warning
DFTR6	DLAT	Warning	Error、Warning
DFTR7	RAM	Warning	Error、Warning、Info、Ignore
DFTR8	clock port	Warning	Error、Warning
DFTR9	DLAT or DFF	Warning	Error、Warning
DFTR10	clock port, DLAT or DFF	Warning	Error、Warning

Rule_ID	检查对象	默认的规则等级	允许指定的规则等级
DFTR11	clock port, DLAT or DFF	Warning	Error、Warning
DFTR12	clock port, DFF	Warning	Error、Warning
DFTR13	clock port, DLAT or DFF	Warning	Error、Warning
DFTR14	clock port, DLAT or DFF	Warning	Error、Warning
DFTR15	clock port, DLAT or DFF	Warning	Error、Warning
DFTR16	clock port, DLAT or DFF	Warning	Error、Warning
DFTR17	DLAT or DFF	Warning	Error、Warning、Info、Ignore
DFTR-TIE0	DLAT or DFF	Warning	Error、Warning、Info、Ignore
DFTR-TIE1	DLAT or DFF	Warning	Error、Warning、Info、Ignore
DFTR-L1	DLAT	Warning	Error、Warning
DFTR-L2	DLAT	Warning	Error、Warning

2.1.3.4 Design Learning

执行 DRC 检查会触发 Design Learning。Design Learning 会根据当前设计中 Cone Analysis 和 Structural Loop Analysis 的结果，识别 SFF 是否为 Scannable Cell。

满足以下条件之一的 SFF 可被识别为 Scannable Cell：

- SE 引脚悬空；
- SE 引脚直连 tie off，如有效值为 1 的 SE 引脚直连了 1’b0 或 tiecell 0；
- SE 引脚直连 Scan 使能信号（通过 **set_scan_signal -type scan_enable** 设置）；
- SE 端口通过组合逻辑连到 Scan 使能信号，且两者极性相同。

说明

- 具有 dont_touch 属性且不在移位寄存器中的 DFF 不参与 DRC 检查。具有 dont_touch 属性且作为移位寄存器单元的 DFF 可忽略其 dont_touch 属性，参与 DRC；
- 被 **set_scan_element false** 命令设定的 Functional FF 不参与 DRC 检查；
- 用户需在 Scan 引擎执行 DRC 及 Design Learning 之前完成对 Cell 的属性设置，包括 dont_touch，dont_use，size_only 等。若在 DRC 之后设置 Cell 的属性，则在执行 **examine_scan_chain** 命令时会自动触发 DRC 检查。

用户可通过 **set_parameter enable_shift_register_identification true** 命令，开启移位寄存器识别功能。之后在执行 **examine_scan_drc** 命令时，Scan 引擎将进行移位寄存器识别。引擎在识别移位寄存器的过程中，会结合 Design Learning 及 DRC 结果，将网表中结构上 QD 相连且时钟相同的寄存器结构标记为移位寄存器。

说明

时序单元组合成移位寄存器需满足以下条件：

- Functional FF 的输出端与另一个 Functional FF 的数据输入端相连，Functional FF 之间仅可用 wire 直连或以穿插 buffer 的方式相连；
- 移位寄存器中被驱动的 FF 不可存在同步 set/reset 的功能性连接。否则该 FF 不能被识别为移位寄存器的非首位；
- 所有 Functional FF 不存在 inferred/DRC_fail/user_defined_nonscannable 属性；
- 所有 Functional FF 属于同一个时钟域及同一个时钟边沿，移位寄存器中遇到不同时钟域或时钟边沿会在边缘处打断，并重新识别移位寄存器，打断后单个 FF 不识别为移位寄存器。
- set_scan_cfg 命令-replace 参数项取值对移位寄存器识别的影响：
- 当该参数项取值为 true 时，Scan 引擎支持将 DFF 识别为移位寄存器的首位；被识别为移位寄存器首位的 DFF，将会在“替换寄存器”和“插入 Scan Chain”的场景中被替换为 SFF。
- 当该参数取值为 false 时，Scan 引擎不支持将 DFF 识别为移位寄存器的首位。

用户可以将全局变量 enable_multibit_shift_register_identification 设置为 true，允许 Multibit Cell 被识别为移位寄存器。仅当 Scan 引擎开启移位寄存器识别功能后，才支持设置该全局变量。Multibit Cell 仅可识别为移位寄存器的首位，其中 Multibit Cell 内最后一位 Bit Cell 作为移位寄存器的首位寄存器，和连接在其后的寄存器组成一组移位寄存器。Multibit Cell 若为多扫描链结构（Multi-Scan Segment），则 Multibit Cell 内任意一位 Bit Cell 都可作为移位寄存器的首位寄存器，和连接在其后的寄存器组成移位寄存器。

用户可通过 **rpt_shift_register** 命令，报告工具识别到的移位寄存器。报告示例如下：

```
rpt_shift_register
Design: demo_case
--------------------------------------------------
InstanceName    ShiftRegID/CellNo
```

```
------------------------------------------------
Sub_a/Ua/SFF0   0/0
Sub_a/Ua/DFF1   0/1
Sub_a/Ua/DFF2   0/2
Sub_a/Ua/DFF3   0/3
Sub_a/Ua/DFF4   0/4
Sub_a/Ub/SFF0   1/0
Sub_a/Ub/SFF1   1/1
Sub_a/Ub/SFF2   1/2
Sub_b/Uc/SFF0   2/0
Sub_b/Uc/SFF1   2/1
Sub_b/Uc/SFF2   2/2
Sub_b/Uc/SFF3   2/3
```

其中，InstanceName 为寄存器单元的实例化名称。ShiftRegID 为移位寄存器编号。CellNo 为该寄存器在其所属移位寄存器组中的级联编号。

2.1.4 预生成及插入 Scan Chain

2.1.4.1 预生成 Scan Chain

用户可通过以下命令分析当前设计中根据用户配置预生成的 Scan Chain：

```
examine_scan_chain
```

该命令会按照配置规则和 Order 算法，将当前设计中的 Scan Cell 及 Wrapper Cell 分配到 Scan Chain 及 Wrapper Chain 上，同时预生成 Scan Chain 及 Wrapper Chain 的链结构。

该命令会对当前设计中的移位寄存器做第三次分析。在此过程中，引擎会基于之前分析的结果，剔除因链长限制而断开生成的单个 Non-Scannable DFF。最终，以第三次分析的结果为依据，形成移位寄存器的识别结果。

用户可通过以下命令查看当前设计中的相关信息：

- **rpt_scan_segment**：报出当前设计中的 Scan Segment 信息。
- **rpt_scan_chain**：报出当前设计中的 Scan Chain 信息。
- **rpt_scan_chain_cell**：报出当前设计下中 Scan Chain 和 Wrapper Chain 上的 Cell 信息。
- **rpt_scan_element**：列出当前设计中所有 Non-Scannable 和 Scannable 对象。

2.1.4.2 插入 Scan Chain

用户可通过以下命令将预生成的 Scan Chain 插入网表：

```
insert_dft_logic
```

该命令会完成实际的替换、插链、以及插入测试逻辑（Lockup Latch/Retiming FF）。Scan 引擎会按照 **examine_scan_chain** 命令中预生成 Scan Chain 及 Wrapper Chain 的链结构，改写网表信息。

2.1.5 输出文件

2.1.5.1 输出插链信息

报告 Chain 的信息

用户可用 **rpt_scan_chain** 命令报出当前设计中的 Scan Chain 和 Wrapper Chain 的详细信息，示例如下：

```
rpt_scan_chain
Design: top
-------------------------------------------------------------------------------------------------------
Chain         Length  Input      Output     ScanEnable  Clocks    Partition         ChainProperty
-------------------------------------------------------------------------------------------------------
I my_chain_0  10      my_si1     my_so1       se    clk0, clk1  Default_Partition   user_defined
I 1           27      test_si1   test_so1     se        clk1    Default_Partition   tool_created
I 2           27      test_si2   test_si2     se        clk1    Default_Partition   tool_created
I 3           27      test_si3   test_so3     se        clk0    Default_Partition   tool_created
I 4           27      test_si4   test_si4     se        clk0    Default_Partition   tool_created
W my_wrp_chain0 15    ext_wSI     ext_wSO  wrp_shift  clk1    Default_Partition   tool_created
W wrp_i_1     28      wrp_si1     wrp_so1  wrp_shift  wrp_clk1  Default_Partition  tool_created
W wrp_i_2     28      wrp_si2     wrp_so2  wrp_shift  wrp_clk1  Default_Partition  tool_created
W wrp_o_1     28      wrp_si3     wrp_so3  wrp_shift  wrp_clk1  Default_Partition  tool_created
-------------------------------------------------------------------------------------------------------
```

用户可用 **rpt_scan_chain** 命令，并结合**-brief** 参数项报出当前设计中的 Scan Chain 和 Wrapper Chain 的简略信息，示例如下：

```
rpt_scan_chain -brief
Design: top
-----------------------------------------------------------------
Chain         Length  Input      Output     ScanEnable  Clocks
-----------------------------------------------------------------
I my_chain_0    10     my_si1    my_so1     se         clk0, clk1
I 1             27     test_si1  test_so1   se         clk1
I 2             27     test_si2  test_si2   se         clk1
I 3             27     test_si3  test_so3   se         clk0
I 4             27     test_si4  test_si4   se         clk0
W my_wrp_chain0 15     ext_wSI   ext_wSO    wrp_shift  clk1
W wrp_i_1       28     wrp_si1   wrp_so1    wrp_shift  wrp_clk1
W wrp_i_2       28     wrp_si2   wrp_so2    wrp_shift  wrp_clk1
W wrp_o_1       28     wrp_si3   wrp_so3    wrp_shift  wrp_clk1
-----------------------------------------------------------------
```

rpt_scan_chain 命令支持报出通过工具其他组件完成 Chain Reorder 处理的 Scan Chain 和 Wrapper Chain 的信息。该功能需搭配**-reordered** 参数项实现，示例如下：

```
rpt_scan_chain -reordered
Design: top
-----------------------------------------------------------------
Chain         Length  Input      Output     ScanEnable  Clocks
-----------------------------------------------------------------
I my_chain_0    15     my_si1    my_so1     se          clk0, clk1
I 1             25     test_si1  test_so1   se          clk1
I 2             26     test_si2  test_si2   se          clk1
I 3             25     test_si3  test_so3   se          clk0
I 4             27     test_si4  test_si4   se          clk0
W my_wrp_chain0 21     ext_wSI   ext_wSO    wrp_shift   clk1
W wrp_i_1       26     wrp_si1   wrp_so1    wrp_shift   wrp_clk1
W wrp_i_2       26     wrp_si2   wrp_so2    wrp_shift   wrp_clk1
W wrp_o_1       26     wrp_si3   wrp_so3    wrp_shift   wrp_clk1
-----------------------------------------------------------------
```

报告的内容说明如下表所示：

表 2-4 Scan Chain 信息说明

表头字段	内容说明
Chain	Chain 的类型与名称，支持类型为 I 和 W。 • I：表示 Scan Chain； • W：表示 Wrapper Chain。
Length	Chain 的长度。
Input	Chain 的数据输入端口名。
Output	Chain 的数据输出端口名。
ScanEnable	Chain 的测试使能信号。
Clocks	Chain 的时钟信号。
Partition	Chain 所属的 Partition。
ChainProperty	Chain 的链属性，支持以下类型：pre_existing、tool_created 以及 user_defined。 • pre_existing：表示用户用 set_scan_chain 指定的 Scan 引擎识别到的设计中存在的 Chain； • tool_created：表示 Scan 引擎新生成的 Chain；

表头字段	内容说明
	• user_defined：表示用户用 add_scan_chain 定义的已有 Chain。

报告 Chain 上 Cell 的信息

用户可使用 **rpt_scan_chain_cell** 命令，报出当前设计中 Scan Chain 和 Wrapper Chain 上 Cell 的详细信息。示例如下：

```
rpt_scan_chain_cell
Design: TOP
-------------------------------------------------------------------------------------------
ChainName     CellNo  InstanceName    Clocks   ClockEdges  RefName  ScanOut ShiftRegID/CellNo
-------------------------------------------------------------------------------------------
I my_chain_0    0  U1/sub_u1_aa/a0/sff0  clk0   (+)  SDFFRPQDX05PT   Q          -/-
                1  U1/sub_u1_aa/a0/sff1  clk0   (+)  SDFFRPQDX05PT   Q          -/-
                2  U1/sub_u1_aa/a0/sff2  clk0   (+)  SDFFRPQDX05PT   Q          -/-
I 1             0  U1/sub_u2_aa/a1/sff0  clk0   (+)  SDFFRPQDX05PT   Q          -/-
                1  U1/sub_u2_aa/a1/sff1  clk0   (+)  SDFFRPQDX05PT   Q          1/0
                2  U1/sub_u2_aa/a1/dff2  clk0   (+)  DFFRPQDX05PT    Q          1/1
                3  U1/sub_u2_aa/a1/dff3  clk0   (+)  DFFRPQDX05PT    Q          1/2
                4  U1/sub_u2_aa/a1/sff2  clk0   (+)  DFFRPQDX05PT    Q          -/-
I 2             0  U1/sub_u2_aa/a2/sff0  clk0   (-)  SDFFRPQDX05PT   Q          -/-
                1  U2/A_multi4bit        clk1   (+)  MB4SDFFRPQDT    Q0         -/-
                2  U2/A_multi4bit:bit1   clk1   (+)  MB4SDFFRPQDT    Q1         -/-
                3  U2/A_multi4bit:bit2   clk1   (+)  MB4SDFFRPQDT    Q2         -/-
                4  U2/A_multi4bit:bit3   clk1   (+)  MB4SDFFRPQDT    Q3         -/-
-------------------------------------------------------------------------------------------
```

用户可用 **rpt_scan_chain_cell** 命令，并结合**-brief** 参数项报出当前设计中 Scan Chain 和 Wrapper Chain 上 Cell 的简略信息，示例如下：

```
rpt_scan_chain_cell -brief
Design: TOP
-----------------------------------------
ChainName     CellNo  InstanceName   Clocks
-----------------------------------------
I my_chain_0    0  U1/sub_u1_aa/a0/sff0  clk0
                1  U1/sub_u1_aa/a0/sff1  clk0
                2  U1/sub_u1_aa/a0/sff2  clk0
I 1             0  U1/sub_u2_aa/a1/sff0  clk0
                1  U1/sub_u2_aa/a1/sff1  clk0
                2  U1/sub_u2_aa/a1/dff2  clk0
                3  U1/sub_u2_aa/a1/dff3  clk0
                4  U1/sub_u2_aa/a1/sff2  clk0
I 2             0  U1/sub_u2_aa/a2/sff0  clk0
                1  U2/A_multi4bit       clk1
                2  U2/A_multi4bit:bit1   clk1
                3  U2/A_multi4bit:bit2   clk1
                4  U2/A_multi4bit:bit3   clk1
-----------------------------------------
```

rpt_scan_chain_cell 命令支持报出通过工具其他组件完成 Chain Reorder 处理的 Scan Chain 和 Wrapper Chain 上 Cell 的信息。该功能需搭配**-reordered** 参数项实现，示例如下：

```
rpt_scan_chain_cell -reordered
Design: TOP
-------------------------------------------------------------
ChainName     CellNo  InstanceName         RefName
-------------------------------------------------------------
I my_chain_0    0  U1/sub_u2_aa/a2/sff0   SDFFRPQDX05PT
                1  U1/sub_u1_aa/a0/sff1   SDFFRPQDX05PT
                2  U1/sub_u1_aa/a0/sff2   SDFFRPQDX05PT
I 1             0  U1/sub_u2_aa/a1/sff0   SDFFRPQDX05PT
                1  U1/sub_u2_aa/a1/sff1   SDFFRPQDX05PT
                2  U1/sub_u2_aa/a1/dff2   DFFRPQDX05PT
                3  U1/sub_u2_aa/a1/dff3   DFFRPQDX05PT
                4  U1/sub_u2_aa/a1/sff2   DFFRPQDX05PT
I 2             0  U1/sub_u1_aa/a0/sff0   SDFFRPQDX05PT
                1  U2/A_multi4bit         MB4SDFFRPQDT
                2  U2/A_multi4bit:bit1    MB4SDFFRPQDT
                3  U2/A_multi4bit:bit2    MB4SDFFRPQDT
                4  U2/A_multi4bit:bit3    MB4SDFFRPQDT
-------------------------------------------------------------
```

rpt_scan_chain_cell 命令报告的内容说明如下表所示：

表 2-5 Scan Chain 和 Wrapper Chain 上的 Cell 信息说明

表头字段	内容说明
ChainName	Chain 的类型及名称。
CellNo	Cell 在 Chain 上的排序。
InstanceName	带层级的实例名称。
Clocks	Cell 的 Scan 时钟信号。
ClockEdges	Cell 的 Scan 时钟极性。
RefName	Cell 的名称。
Scan_out	Cell 的输出引脚类型。
ShiftRegID/CellNo	移位寄存器的标识序号。例如，1/2 表示第一组移位寄存器中的第二位寄存器。

报告插链网表的寄存器信息

rpt_insertion_info 命令支持报出插链完成后，网表内寄存器数量的详细信息，示例如下：

```
rpt_insertion_info
Design: demo_case
-----------------------------------------------------------------
No.       Item                                  Quantity
-----------------------------------------------------------------
1         Total Flip-Flop (FF) Count            400
2         Single Data Flip-Flop (DFF) Count     200
3         Multi-bit DFF Count                   25
4         Multi-beat DFF Count                  -
5         Single Scan Data Flip-Flop (SFF) Count 100
6         Multi-bit Single Scan In/Out SFF Count 25
7         Multi-bit Multi Scan In/Out SFF Count 50
8         Multi-beat SFF Count                  -
9         Scannable FF Count                    350
10        Nonscannable FF Count                 50
11        Total FF Bit Count                    600
12        Scannable FF Bit Count                500
13        Nonscannable FF Bit Count             100
14        Scan Chain Cell Count                 250
15        Wrapper Chain Cell Count              100
16        Shared Wrapper Cell Count             60
17        Dedicated Wrapper Cell Count          40
18        Scan Chain Cell Bit Count             400
19        Wrapper Chain Cell Bit Count          100
20        Shared Wrapper Cell Bit Count         60
21        Dedicated Wrapper Cell Bit Count      40
-----------------------------------------------------------------
```

报出的寄存器信息包括：

表 2-5 rpt_insertion_info 报告的寄存器信息说明

字段	内容说明
Total Flip-Flop (FF) Count	FF 总数。
Single Data Flip-Flop (DFF) Count	单比特 DFF 数量。
Multi-bit DFF Count	多比特 DFF 数量。
Multi-beat DFF Count	多拍 DFF 数量。
Single Scan Data Flip-Flop (SFF) Count	单比特 SFF 数量。

字段	内容说明
Multi-bit Single Scan In/Out SFF Count	多比特单输入输出 SFF 数量。
Multi-bit Multi Scan In/Out SFF Count	多比特多输入输出 SFF 数量。
Multi-beat SFF Count	多拍 SFF 数量。
Scannable FF Count	Scannable FF 数量。
Nonscannable FF Count	Nonscannable FF 数量。
Total FF Bit Count	FF 总比特数。
Scannable FF Bit Count	Scannable FF 比特数。
Nonscannable FF Bit Count	Nonscannable FF 比特数。
Scan Chain Cell Count	Scan Chain 内 Cell 总数。
Wrapper Chain Cell Count	Wrapper Chain 内 Cell 总数。
Shared Wrapper Cell Count	Shared Wrapper Cell 数量。
Dedicated Wrapper Cell Count	Dedicated Wrapper Cell 数量。
Scan Chain Cell Bit Count	Scan Chain 内 Cell 比特数。
Wrapper Chain Cell Bit Count	Wrapper Chain 内 Cell 比特数。
Shared Wrapper Cell Bit Count	Shared Wrapper Cell 比特数。
Dedicated Wrapper Cell Bit Count	Dedicated Wrapper Cell 比特数。

2.1.5.2 输出网表

用户可通过 **dump_netlist** 命令输出插链后的网表。示例如下：

```
dump_netlist -file "./xxx.v"
```

2.1.5.3 输出 CTL 文件

用户可通过 **dump_ctl** 命令输出 CTL 文件。示例如下：

```
dump_ctl "./xxx.ctl"
```

2.1.5.4 输出 SCANDEF 文件

插链完成后，Scan 引擎将按照内置的顺序规则生成 SCANDEF 文件，用来描述 Scan Cell 的顺序。默认情况下，描述的对象为当前设计中 Scan Chain 上的所有 Scan Cell。

用户可通过 **dump_def** 命令输出 SCANDEF 文件。示例如下：

```
dump_def -section scan_chain -file xxx.scandef
```

SCANDEF 文件的内容分为两部分：

- 无序部分（FLOATING）：包含未指定顺序的 Scan Cell。
- 有序部分（ORDERED）：包含已指定顺序的 Scan Cell，作为整体参与和 FLOATING 部分的重排序。

Scan Cell 包括单 Bit 和多 Bit 两种类型，并以实例名称的方式列举。多 Bit 的 Scan Cell 需以 BITS 表示 Scan Cell 内部 Bit 的数量。BITS 为可选项，默认为 1。

同时满足相同时钟域和时钟边沿的 Cell Group 视为一个 Partition，partition_name 仅用于匹配的标识。不同 Partition 之间的 Scan Cell 不参与重排序。

Scan 引擎将 Wrapper Cell 和 Scan Cell 分配在不同的 Partition 中。Scan Cell 分配在 ORDERED 部分的情况有：

- Scan Cell 为移位寄存器；
- Scan Cell 为用户以 **set_scan_segment** 的方式定义的 Scan Segment；
- 定义在 CTL 文件中的 Scan Cell。

Scan 引擎支持导出经过预处理的 SCANDEF 文件，以提高后续集成操作的便利性。命令示例如下：

```
trim_scan_chain
dump_def -section scan_chain -file test_case.scandef
```

在未经过预处理的 SCANDEF 文件中，START 和 STOP 字段会展示 Scan Chain 的扫描数据输入输出端口，不便于后续的 EDT 集成等操作。通过添加**-trim** 参数，Scan 引擎会对 SCANDEF 文件进行预处理。在处理后的文件中，START 和 STOP 字段会展示 Scan Chain 首尾的寄存器信息，便于后续的 EDT 集成等操作。

2.2 Scan 引擎支持场景及 Dofile 示例

2.2.1 替换寄存器

Scan 引擎支持在 Scan Chain 插入前进行寄存器的替换。该场景在不执行 DRC 的情况下，仅将网表中的 DFF 替换为 SFF，而不进行 Scan Chain 插入。用户可使用如下命令进行寄存器替换：

```
set_scan_cfg -replace true
insert_dft_logic -replace_only
```

下表介绍了寄存器的替换规则：

表 2-15 寄存器替换规则

DFF 位置	规则说明
非移位寄存器中的 DFF	替换为对应的 SFF
移位寄存器中的 DFF	• 位于首位的 DFF：替换为对应的 SFF • 位于非首位的 DFF：保持原有结构，不做替换处理

替换寄存器的流程如下：

```
# load CTL test model and library and the synthesized netlist
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib
load_netlist ./inputs/pre_scan_demo.v

# set current working design
present_design demo_top

# configure scan for current design
set_scan_cfg -replace true

# replace ff
insert_dft_logic -replace_only

# Write out the scan-replaced design
dump_netlist -file  "../outputs/post_replace_demo.v"
```

Scan 引擎支持将网表中的用户指定 SFF 替换为 DFF，而不进行 Scan Chain 插入。用户可使用如下命令进行 SFF 寄存器的替换：

```
insert_dft_logic -replace_unscan
```

替换寄存器的流程如下：

```
# load CTL test model and library and the synthesized netlist
```

```
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib
load_netlist ./inputs/pre_scan_demo.v

# set current working design
present_design demo_top

# configure scan for current design
set_scan_cfg -replace true
set_scan_element false {Ua/U1/sff0 Ub/U1/sff0 Ua/U1/MB4SFF0 Ua/U1/MB4SFF1:bit0 Ua/U1/MB2SFF0:bit1}
insert_dft_logic -replace_unscan

# replace ff
insert_dft_logic -replace_only -replace_unscan

# Write out the scan-replaced design
dump_netlist -file  "../outputs/post_replace_demo.v"
```

说明

在完成该流程后可继续进行其他流程，包括仅连接 ICG、Scan Chain 插入。

2.2.2 仅连接 ICG

Scan 引擎支持单独对设计中的 Clock-Gating 配置控制信号。

用户可以使用以下命令对符合条件的 Clock-Gating 做单独连接，将 Clock-Gating 的控制引脚连接到指定 scan_enable 信号。示例如下：

```
insert_dft_logic -connect_icg_only
```

该命令会将设计中符合条件的 Clock-Gating 的控制引脚（除 Function Enable 之外的 Test Enable 或 Scan Enable）断开原来连接，并重新连接到用户定义的 scan_enable 信号上。以下两种情况会保留原来连接：

- 不符合条件的 Clock-Gating 的 TE 端保留原来连接；
- Clock-Gating 的 Function Enable 引脚保留原来连接。

符合条件的 Clock-Gating 指带有 insert_icg 属性，且控制信号连接情况为以下情况之一：

- 悬空；
- 直连 1'b0；
- 通过若干 buffer 连到 1'b0，或通过奇数个 inverter 连到 1'b1；
- 经过 inverter/buffer 后悬空。

用户定义的 scan_enable 信号指用户通过以下命令定义给 Clock-Gating 专用的 scan_enable 信号：**set_scan_signal -view spec -type scan_enable -usage clock_gating** 或 **all**。

如果用户通过以下命令定义了多个已有的 scan_enable 信号，Scan 引擎会随机取一个：**set_scan_signal -view existing -type scan_enable -usage clock_gating** 或 **all**。

如果用户使用 **set_scan_signal** 命令为 scan_enable 信号指定了 hookup_pin，则给 Clock-Gating 指定的 scan_enable 信号将连接在该 hookup_pin 上。

若用户没有定义任何 scan_enable 信号，Scan 引擎将自动创建一个 scan_enable 信号，并定义在新创建的端口上。端口名称默认为 test_se。完成配置 Clock-Gating 后，在 Scan Chain 插链过程中需重新定义该 scan_enable 信号。

单独对 Clock-Gating 配置控制信号的仅连接 ICG 的流程如下：

```
# load CTL test model and library and synthesized netlist
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib
load_netlist ./inputs/pre_scan_demo.v

# Set current working design
present_design demo_top

# Configure scan enable signal for current design
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0 -usage clock_gating -view spec

# Insert scan chains and report corresponding results
insert_dft_logic -connect_icg_only

# Write out the icg-connected design
dump_netlist -file  "../outputs/post_scan_demo.v"
```

说明

在完成该流程后可进行继续其他流程，包括替换寄存器以及 Scan Chain 插入。

2.2.3 插入 Scan Chain

Scan 引擎支持对设计进行 Scan Chain 插入，以下 Scan Chain 插入示例中分别为基本 Scan Chain 插入场景、读取 CTL 后进行 Scan Chain 插入场景以及分不同 DFT Partition 进行 Scan Chain 插入场景。

用户可使用如下命令进行 Scan Chain 插入：

```
insert_dft_logic
```

说明

- 用户执行 **insert_dft_logic** 命令后，网表已被修改，后续执行命令的对象为修改后的设计。
- 仅支持用户对当前设计执行 Scan Insertion，不支持用户在执行 **insert_dft_logic** 命令之后对其它设计执行 Scan Insertion 操作。

2.2.3.1 基本场景

Scan 引擎支持对设计进行插链，基本场景的流程如下所示：

```
# load libraries
load_lib  {../libs/demo1.lib ../libs/demo2.lib}

# load the synthesized netlist
load_netlist ../inputs/IPBLK.v -top core

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK} -off_state 0
set_scan_signal -type wrapper_clock -port WRCK -off_state 0
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0
set_scan_signal -type constant -port C1 -constant_value 1

# Configure wrapper
set_wrapper_cfg enable -chain_count 2 -style shared -reuse_threshold 16 \
-dedicated_cell_type WC_D1
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}

# Configure scan
set_scan_cfg -max_length 300 -mix_edges true -mix_clocks true -replace false

# DRC
examine_scan_drc -verbose
examine_scan_drc -verbose -file ./result/drc_result_file

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic

# Write out the CTL test model, SCANDEF file, and scan-inserted design
dump_ctl  "./IPBLK.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/IPBLK_scan.v"
```

2.2.3.2 读取 CTL

Scan 引擎支持对具有层级结构的设计进行扫描链插入。完成 Core 级别的扫描链插入后，需在 Top Level 下读取对应的 Core-Level CTL 文件和库文件，继而执行顶层扫描链插入。流程如下所示：

```
# load libraries
```

```
load_lib  {../libs/demo1.lib ../libs/demo2.lib ../libs/IPBLK.lib}

# load the synthesized netlist
load_ctl -module module1 ../inputs/IPBLK.ctl
load_netlist ../inputs/top.v

# Set current working design
present_design demo_top

# Define pre-existing scan chain and scan segment
set_scan_segment seg1 -access {scan_enable /U1/SE scan_data_in /U1/SI \
scan_data_out /U7/SO} -lockup_exists false
set_scan_chain my_chain0 -scan_enable SE -scan_data_in SI -scan_data_out SO -class scan
set_scan_chain my_wrp_chain1 -scan_enable wrp_shift -scan_data_in ext_SI \
-scan_data_out ext_SO -class wrapper

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK} -off_state 0
set_scan_signal -type wrapper_clock -port WRCK -off_state 0
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0
set_scan_signal -type constant -port C1 -constant_value 1

# Configure wrapper chain
set_wrapper_cfg enable -chain_count 2 -style shared -reuse_threshold 16 \
-dedicated_cell_type user_defined
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}
add_dedicated_wrapper_cell_type -design_name MY_INPUT_WRAPPER_CELL -interface \
{shift_clk subsys_clk h capture_en capture_dr h shift_en shift_dr h cti si h cto so \
h cfi data_in h cfo data_out h}

# Configure scan chain
set_scan_cfg -max_length 300 -mix_edges true -mix_clocks true -replace false
add_scan_chain my_add_chain0 -ordered_elements {dff1 dff2 dff3} -scan_enable SCAN_EN -scan_data_in
SI2 -scan_data_out SO2

# DRC
examine_scan_drc

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic

# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl  "./top.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
dump_netlist -file  "../outputs/top.v"
```

2.2.3.3 DFT Partition

Scan 引擎支持分 DFT Partition 进行插链配置。允许用户划分 DFT Partition 来分别配置 Scan Chain，并允许在各 Partition 层级下配置 Wrapper Chain。

分不同 DFT Partition 进行 Scan Chain 插入的流程如下所示:

```
# load CTL test model and library
load_ctl -module core1 ./inputs/pre_scan_demo.ctl
load_lib ../libs/demo.lib

# load the synthesized netlist
load_netlist ./inputs/pre_scan_demo.v

# Set current working design
present_design demo_top

# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK} -off_state 0
set_scan_signal -type wrapper_clock -port WRCK -off_state 0
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0

# Configure wrapper
set_wrapper_cfg enable -chain_count 2 -style shared -reuse_threshold 16 \
-dedicated_cell_type user_defined
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}
add_dedicated_wrapper_cell_type -design_name MY_INPUT_WRAPPER_CELL -interface \
{shift_clk subsys_clk h capture_en capture_dr h shift_en shift_dr h cti si h cto so \
h cfi data_in h cfo data_out h}

# Configure partition
add_scan_partition p1 -clocks CLK
add_scan_partition p2 -clocks RXCLK

# Configure scan for partition p1
set_current_scan_partition p1
set_scan_cfg -max_length 250 -mix_edges true -mix_clocks true

# Configure scan for partition p2
set_current_scan_partition p2
set_scan_cfg -max_length 200 -mix_edges true -mix_clocks true

# Configure scan for default partition
set_current_scan_partition Default_Partition
set_scan_cfg -max_length 300
set_scan_chain my_chain1 -scan_enable SCAN_EN \
-scan_data_in SI1 -scan_data_out SO1 -class scan

# DRC
examine_scan_drc

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains
insert_dft_logic

# Write out the CTL test model, SACNDEF file and scan-inserted design
dump_ctl  "./post_scan_demo.ctl"
dump_def -section scan_chain -file post_scan_demo.scandef
```

```
dump_netlist -file "../outputs/post_scan_demo.v"
```

2.2.4 对多个设计进行 Scan 插入并集成

Scan 引擎支持对多个独立设计进行 Scan 插入，包括：设置 Scan 参数、替换寄存器、连接 ICG、插入 Scan Chain。

如下流程展示了分别一次对 module0、module1 和 module2 进行 Scan 配置和 Scan 插入，最终将 module0 和 module1 实例化至 module2 的场景：

```
# load libraries
load_lib {../libs/demo1.lib ../libs/demo2.lib}

# load the synthesized netlist
load_netlist { ../inputs/case0.v ../inputs/case1.v ../inputs/case2.v }

# case0 scan
source "case0.tcl"

# case1 scan
source "case1.tcl"

# case2 scan
source "case2.tcl"

# integrate module1 and module2 to module0
present_design module0
add_inst U1 module1
add_inst U2 module2
# ECO operation
connect_pin -from clk0 to { U1/clk0 U2/clk0 } -port_name clk0
…

dump_netlist -file "../outputs/integrated_case0.v"

# Scan script. case1.tcl and case2.tcl are similar to case0.tcl.
##--------------------case0.tcl--------------------##
# replace
present_design module0
insert_dft_logic -replace_only

# connect ICG
set_scan_signal -type scan_enable -port ICG_SE -off_state 0 -usage clock_gating
insert_dft_logic -connect_icg_only

# Scan Chain stitch
# Define DFT signals
set_scan_signal -type clock -port {CLK RXCLK TXCLK} -off_state 0
set_scan_signal -type wrapper_clock -port WRCK -off_state 0
set_scan_signal -type scan_enable -port SCAN_EN -off_state 0 -usage scan
set_scan_signal -type constant -port C1 -constant_value 1

# Configure wrapper
```

```
set_wrapper_cfg enable -chain_count 20 -style shared -reuse_threshold 32 \
-dedicated_cell_type WC_D1
set_wrapper_cfg -style none -port {WSI WSO WRCK ShiftWR CaptureWR}

# Configure scan
set_scan_cfg -max_length 300 -mix_edges true -mix_clocks true -replace false

# DRC
examine_scan_drc -verbose
examine_scan_drc -verbose -file ./result/drc_result_file0

# Analyze and preview the different scan chains before stitching the chains
examine_scan_chain

# Insert scan chains and report corresponding results
insert_dft_logic

# Write out the CTL test model, SCANDEF file, and scan-inserted design
dump_ctl  "./case0.ctl"
dump_def -section scan_chain -file post_scan_case0.scandef
dump_netlist -file  "../outputs/post_scan_case0.v"
```

3 附录

3.1 术语表

3.1 特殊场景说明

3.1.1 Wrapper 分析与用户自定义 Chain 存在冲突时的处理策略说明

Scan 引擎支持提供 add_scan_chain 命令添加用户自定义 chain；然而，在用户同时开启生成 Wrapper Chain 的功能时，可能存在冲突情况：Wrapper 分析到的目标 Shared Wrapper Cell 可能已经被用户通过 add_scan_chain 命令指定作为自定义 Chain 的一部分。

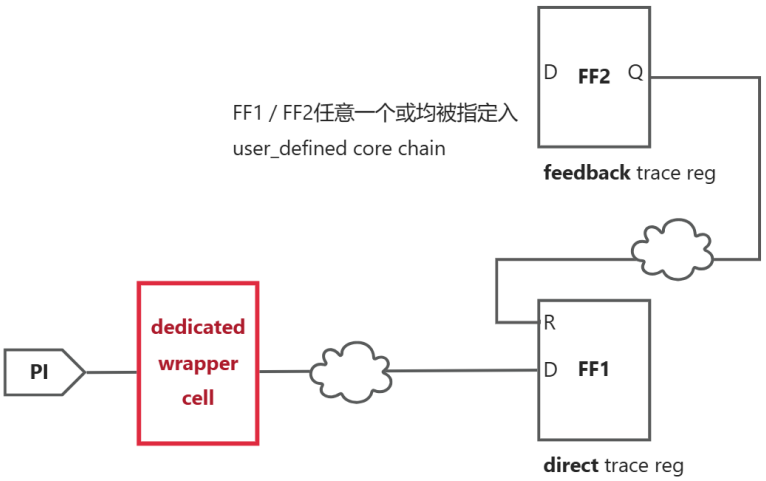
针对这种冲突场景，Scan 引擎支持通过命令配置应对策略。如下所示：

```
add_scan_chain -priority_with_wrapper priority_value
```

其中 *priority_value* 为用户可选的发生冲突时的策略，存在 3 个可选值：**dedicated_first、internal_first** 和 **shared_first。**

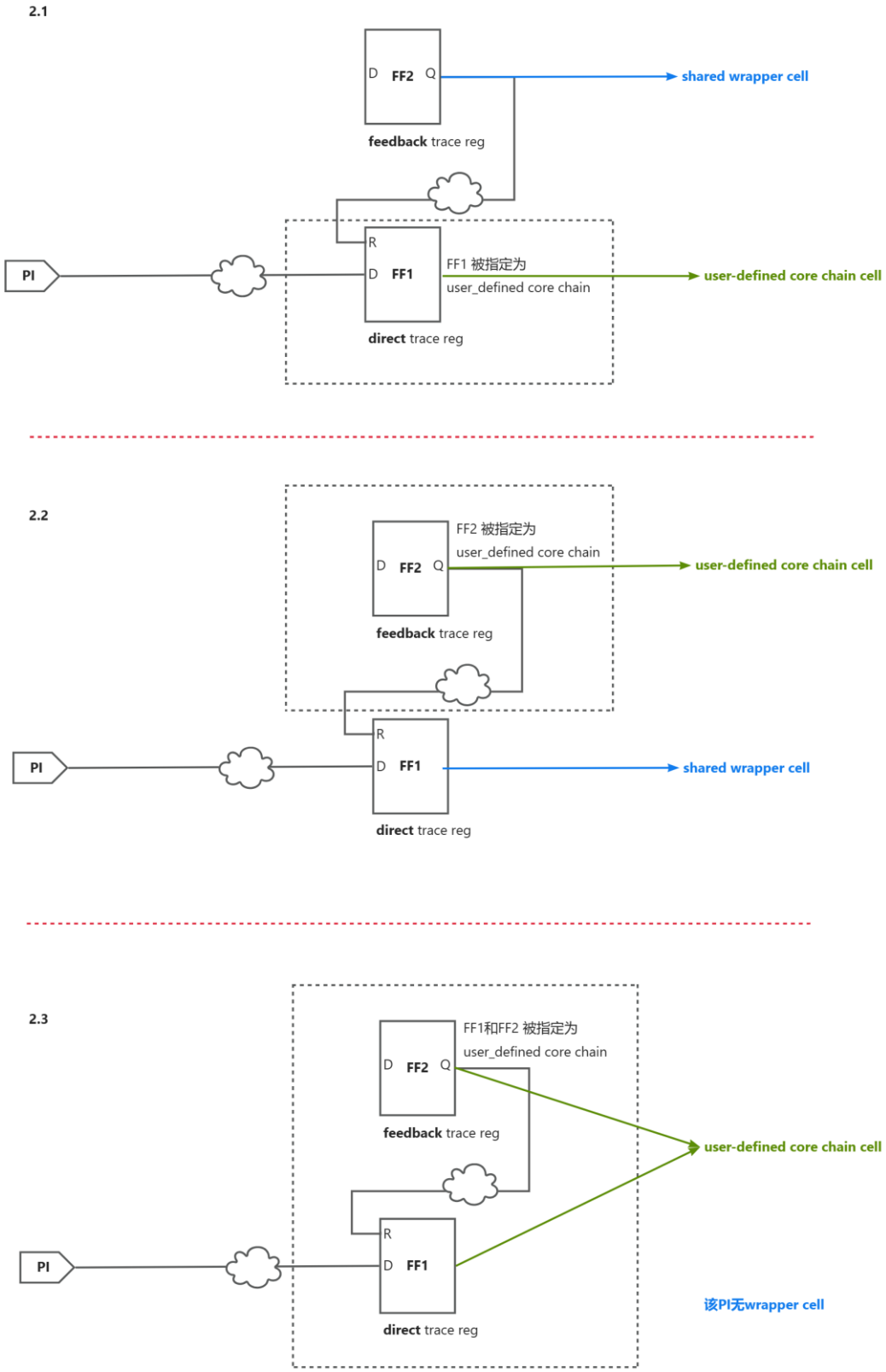
1. **dedicated_first：**若对某个 PI/PO 端口进行 Wrapper 分析，且追踪的目标 Shared Wrapper Cell（无论是 Derict 追踪还是 Feedback 追踪到的目标单元）已被 add_scan_chain 命令指定为自定义 Chain 的一部分，则 Scan 引擎直接对该端口插入一个 Dedicated Wrapper Cell。如图 3-1 所示。

   图 3-1 dedicated_first 策略示意图



2. **internal_first：**若对某个 PI/PO 端口进行 Wrapper 分析，且追踪的目标 Shared Wrapper Cell（无论是 Derict 追踪还是 Feedback 追踪到的目标单元）已被 add_scan_chain 命令指定为自定义 Chain 的一部分，则这些 Cell 会参与自定义 Chain，而不会作为 Shared Wrapper Cell 参与 Wrapper Chain。如图 3-2 所示。

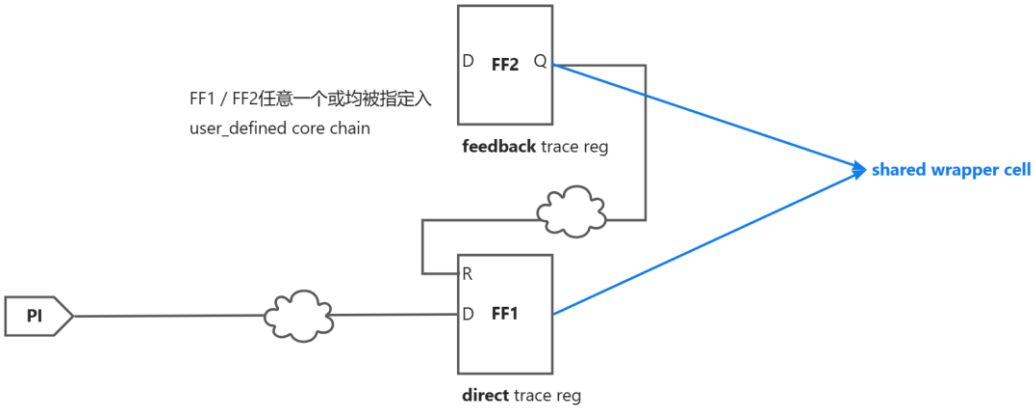
   图 3-2 internal_first 策略示意图



3. **shared_first：**若对某个 PI/PO 端口进行 Wrapper 分析，追踪的目标 Shared Wrapper Cell（无论是 Derict 追踪还是 Feedback 追踪到的目标单元）被 add_scan_chain 命令指定为自定义 Chain 的一部分，则这些 Cell 会作为 Shared

Wrapper Cell 参与 Wrapper Chain。同时，add_scan_chain 命令指定的自定义 Chain 将不再包含这些 Cell。如图 3-3 所示。

图 3-3 shared_first 策略示意图



以上三种处理策略的优劣情况总结如下：

表 3-1 Wrapper 分析与用户自定义 chain 存在冲突时的处理策略说明

策略名称	dedicated_first	internal_first	shared_first
策略说明	存在冲突时，优先满足用户自定义 Chain；端口直接插入 Dedicated Wrapper Cell。	存在冲突时，优先满足用户自定义 Chain；端口不插入 Dedicated Wrapper Cell。	存在冲突时，冲突 Cell 优先作为 Shared Wrapper Cell；用户自定义 Chain 将不再包含冲突 Cell。
优点	1. 保证了 Wrapper 功能的正确性。 2. 完全满足用户自定义 Chain。	1. 完全满足用户自定义 Chain。 2. 不新增 Dedicated Wrapper Cell 的数量，面积友好。	1. 保证了 Wrapper 功能的正确性。 2. 不新增 Dedicated Wrapper Cell 的数量，面积友好。
缺点	增加 Dedicated Wrapper Cell 的数量，导致设计面积增大。	不能保证 Wrapper 功能的正确性。	不能完全满足用户自定义 Chain。

3.2 术语表

3.2.1 A-E

Associated Internal Clocks

一种定义在引脚（Pin）上的一个或一组内部时钟信号，与定义在端口（Port）上的源时钟的时钟波形完全一致。

ATPG

Automatic Test Pattern Generation，自动测试向量生成，用于生成测试向量以检测集成电路中的缺陷。

Black Box

黑盒，指在设计中没有详细描述内部结构的模块。

Bi-directional Port

双向端口，指可以同时作为输入和输出的端口。

Cell Library

单元库，包含标准单元的功能和时序信息。

Clock Domain

时钟域，指由同一时钟信号驱动的逻辑单元集合。

Clock Edge

时钟边沿，指时钟信号的上升或下降沿。

Clock-Gating

一种通过关闭未使用电路的时钟来降低功耗的门控时钟单元。

Clock Cone

时钟锥，指一个时钟信号影响的所有逻辑单元的集合。

Combination Logic

组合逻辑，指不包含时序单元的逻辑电路。

Core

一种已插入扫描链结构的设计电路模块。

CFI

Core Functional Input，对于输入的 Wrapper Cell，该信号来自 Core 周围的逻辑；对于输出的 Wrapper Cell，该信号来自 Core。

CFO

Core Functional Output，对于输入的 Wrapper Cell，该信号驱动 Core；对于输出的 Wrapper Cell，该信号驱动 Core 周围的逻辑。

CTI

Core Test Input，Wrapper Cell 的测试输入，该信号可以来自 Primary Input（如果该 Wrapper Cell 处于 Wrapper Chain 的首位）或者 Wrapper Chain 上前一个 Wrapper Cell 的 CTO 信号。

CTO

Core Test Output，Wrapper Cell 的测试输出，该信号可以驱动 Primary Output（如果该 Wrapper Cell 处于 Wrapper Chain 的末位）或者 Wrapper Chain 上后一个 Wrapper Cell 的 CTI 信号。

CTL

Core Test Language，为 IEEE 1450.6 协议，是一种用于描述和执行集成电路（IC）测试的标准化语言，一种对 STIL 协议（IEEE 1450）的扩展，提供了统一的测试描述、配置、执行和数据管理方式。

DFF

Data Flip-Flop，一种具有存储功能的 D 型触发器。

DFT Partition

允许将设计电路中的时序单元指定在多个独立的分区（Partition）中，以便于扫描链可在各自独立的分区中插入。

DRC

Design Rule Checking，设计规则检查。根据设计规则检查设计电路中是否存在违反这些规则的电路结构，确保良好的可测试性设计。

DRC Debugging

DRC 调试，指通过分析 DRC 违例信息来定位和修复设计中的问题。

Dofile

脚本文件，用于自动执行一系列命令。

EXTEST Mode

外向测试模式，是数字集成电路中设计可测试性（DFT）技术的一部分，专门用于测试核心（Core）外部的逻辑电路。在外向测试模式下，使用封装链（Wrapper Chain）控制和观察信号：其中输出封装链（Output Wrapper Chain）用来发送测试向量；输入封装链（Input Wrapper Chain）用来捕获数据。这个模式的主要目的是在不依赖核心内部逻辑的情况下，验证核心外围的互连和外部逻辑的正确性。

3.2.2 F-J

Fan-in

扇入，指输入到一个逻辑门的信号数量。

Fan-out

扇出，指从一个逻辑门输出的信号数量。

FLOATING

指 SCANDEF 文件中扫描链上可以调换顺序的扫描单元部分。

Functional FF

Functional Flip-Flop，功能触发器，指存在于设计中用于实现逻辑功能的触发器，包括 Data Flip-Flop（DFF）和 Scan Data Flip-Flop（SFF）。

Gate-level Netlist

门级网表，描述电路中逻辑门的连接关系。

Hierarchical DFT Insertion

该术语涉及如何在工具中对层次化电路结构的设计执行扫描链分层插入的方法，即在底层设计模块中执行扫描链插入，然后将该模块的扫描结构合并到较高层的设计中。

ICG

Integrate Clock Gating，集成门控时钟单元，一种通过时钟使能信号来控制后续模块的时钟信号。ICG 是一种低功耗设计中使用的技术，它可以通过关闭未使用电路的时钟来降低功耗。

Instance

实例化，指设计中的一个具体的逻辑单元或模块。

Internal Clock

一种扫描时钟，定义在设计中的内部引脚（Pin）上。内部时钟可以由工具在逻辑门输出处自动确定，也可以在时钟网络内的相关内部时钟引脚处手动指定。

3.2.3 K-O

Latch

锁存器（Latch）是一种对脉冲电平敏感的存储单元电路，可以在特定输入脉冲电平作用下改变状态。锁存，就是把信号暂存以维持某种电平状态。锁存器的最主要作用是缓存，其次是解决高速控制器与慢速外设的不同步问题，再其次是解决驱动的问题，最后是解决一个 I/O 口既能输出也能输入的问题。

LE

LE（Leading Edge）是指时钟信号波形在一个周期内的首个沿。对于初始状态值为 0 的时钟，首个沿为上升沿；对于初始状态值为 1 的时钟，首个沿为下降沿。时钟信号波形的首个沿所能触发的寄存器具有 LE 属性。

Liberty

一种描述标准单元库的格式，包含单元的功能、时序和功耗等信息。

Lockup Latch

Lockup Latch 指的是用于在扫描链中防止时序问题的锁存器。

Max Logic Level

最大逻辑深度，指输入端口到第一个存储单元之间的最大组合逻辑层数。

Module

一种包括模块声明、端口声明、内部信号声明和功能实现的电路模块，是定义硬件设计的基本功能单元，可将复杂的电路设计分解为更小、更易管理的模块。

Non-Scannable

一种时序单元的属性，被标记为该属性的时序单元不可插入扫描链中，不可作为扫描链上的时序单元。

ORDERED

指 SCANDEF 文件中扫描链上不可调换顺序的扫描单元部分。

3.2.4 P-T

Pre-existing Scan Info

已存在于设计电路中的扫描结构，包括扫描段（Scan Segment）和扫描链（Scan Chain）。

Pseudo Input

伪输入，指将内部信号转化为逻辑上可直接控制的输入信号。

PI

Primary Input，主要输入端口，指设计的主要输入信号端口。

PO

Primary Output，主要输出端口，指设计的主要输出信号端口。

Pin

引脚，指逻辑单元的输入或输出端口。

RAM

Random Access Memory，随机存取存储器，指一种可读写存储器件。

Reuse Threshold

复用阈值，指设置共享单元的复用阈值。

Scan Cell

一种同时具有功能和扫描两种操作模式的时序单元，在该工具中仅指多路复用触发器（Multiplexed-D Flip-Flop）结构。

Scan Chain

扫描链是一系列具有功能和扫描两种操作模式的触发器通过串行连接形成的一个或多个链。扫描链在测试模式下可以将数据移入和移出电路，从而检测电路内部逻辑的故障。

Scan Segment

扫描段是较底层级别实例或子模块内的扫描链的一部分，由电路设计中的一系列连续的时序单元组成，其中上一级时序单元的数据输出端口（Q）和下一级时序单元的测试数据输入端口（SI）直接相连。

Scan Partition

进行单独 Scan Configuration 的分区，用户可以通过添加不同的 Scan Partition 进行单独的配置。

SCANDEF

一种描述扫描链结构的 DEF 文件，布局布线工具可基于该 DEF 文件对扫描链上的时序单元进行重新排序和重新分区。

Scannable

一种时序单元的属性，被标记为该属性的时序单元可插入扫描链中，可作为扫描链上的时序单元。

Sequential Cell

一种存在于电路设计中的具有存储功能的时序单元，其存储值不仅依赖于当前输入信号，还依赖于上一刻状态的输入信号。常见类型包括触发器（Flip-Flops）、锁存器（Latch）、随机存取存储器(RAM)。

SFF

Scan Data Flip-Flop，一种同时具有功能性和扫描性两种操作模式的 D 型触发器。

Shared Cell

指 Wrapper Chain 上可复用的电路中原有的可扫描单元，作为封装单元使用。

Shift Register

移位寄存器由电路设计中的一系列连续时序单元组成，其中上一级时序单元的数据输出端口（Q）和下一级时序单元的功能数据输入端口（D）直接相连。移位寄存器只需更换其首位寄存器即可插入到扫描链中。

System Clock

系统时钟，指设计中用于功能模块的主时钟信号。

TE

Trailing Edge，时钟信号波形在一个周期内的最后一个沿。对于初始状态值为 0 的时钟，最后一个沿为下降沿；对于初始状态值为 1 的时钟，最后一个沿为上升沿。时钟信号波形的最后一个沿所能触发的寄存器具有 TE 属性。

Test Clock

测试时钟，指用于测试模式下的时钟信号。

Tie-cell

绑定单元，指用于将信号固定为高或低电平的单元。

3.2.5 U-Z

Verilog

一种硬件描述语言，用于描述电路的结构和行为。

Waveform

波形，指信号随时间变化的图示。

Wrapper Chain

针对面积大、电路复杂的芯片，为了增加可预测性，通常需要应用分层技术（Hierarchical Techniques）。Wrapper Chain 是应用较为广泛的技术之一，特别是针对可分割成多个 block 的大芯片。Wrapper Chain 针对 block level 可控制性差的 Input 和可观测性差的 Output，将与之相连的时序单元替换为 Scan Cell 后串成 Input/Output Wrapper Chain。

Wrapper Cell

构成 Wrapper Chain 的基本单元，包括 Dedicated 和 Shared 两种。Dedicated Wrapper Cell 会增加额外的 D 型触发器，而 Shared Wrapper Cell 复用电路中已有的 D 型触发器。

Wrapped Core

添加 Wrapper Chain 后，可以使用在 Intest Mode 下生成的测试向量单独进行 ATPG 仿真和 WP/FT 测试的逻辑，通常为一个功能独立的 Block。